探討如何建立一個分析 **AI** 語義空間強吸引子碰撞後圖譜的 **Agent Skill**

回到上面第一個 **Prompt** 的問題

“

附件 tar 的建立次序是《𝕆 → G2_SO(4) → ℍ → ℂ² 成界過程初探 1-23》、《與 ChatGPT 5.6 探討「SMFT」、「成界之學」開宗立派還欠缺的準備材料》然後是四份英文綱領。。。

”

我發現這樣寫出來的文章，

《From Dialogue to Research Architecture: How Long-Horizon Human–AI Collaboration Revises, Filters, and Distills Theory - A Case Study in Adaptive Semantic Collision, Reconstructable Research, and Human-Governed Search-Space Formation》
https://osf.io/kcjv3/files/osfstorage/6ab90bf3c95c0022bb3c39b1

《From Dialogue to Research Architecture — Short Version》
https://osf.io/kcjv3/files/osfstorage/6a5419fe98507cd2fa7afc0a

目前的讀者生態很難傳播出去。我開始懷疑，也許可以做一個 **GIT**，讓別人以

《The_Semantic_Collider》
https://osf.io/kcjv3/files/osfstorage/6a785b939547f3b9621fb592
和《Reconstructable Research》
https://osf.io/kcjv3/files/osfstorage/6a78fb1ab195de03f21fb7bb

的框架，模仿分析自己類似 𝕆 → **G2_SO(4)** → ℍ → ℂ² 成界過程初探 **1-23**》的人-機 互動推演。這可以有兩個落實方向：

1）基本上就是上傳以上文件到 **GIT**，讀者下載全套文件，配合一個 **Skill** 模仿重複的分析，或解答相關問題。例如，這個分析是如何做的？要注意什麼？以至 **baseline** 的文件 《𝕆 → **G2_SO(4)** → ℍ → ℂ²》是關於什麼等。。。

2）從新寫一個 **kernel** 給《**The_Semantic_Collider**》、《**Reconstructable Research**》、《**From Dialogue to Research Architecture: How Long-Horizon Human–AI Collaboration Revises, Filters, and Distills Theory**》、《**From Dialogue to Research Architecture — Short Version**》作為 **Skill**，《𝕆 → **G2_SO(4)** → ℍ → ℂ² 成界過程初探 **1-23**》是一份 **Sample raw data**（也同時幫我宣傳這個框架）。

《與 **ChatGPT 5.6** 探討「**SMFT**」、「成界之學」開宗立派還欠缺的準備材料》和四份英文綱領，則再設計其在上面計劃中的相關角色。

你覺得對於傳播 **SMFT**、成界之學的幫助，會有根本性的改變嗎？

我認為會有根本性的改變，但這個「根本性」主要發生在傳播機制，而不是立即提升 **SMFT**／

成界之學的科學地位。

現在的傳播模式基本上是：

> 我有一套理論 → 請你讀文章 → 理解它 → 判斷它是否值得相信。

你現在提出的 Git + Skill 模式可以變成：

> 我有一個可執行的研究方法 → 你拿自己的 **Human–AI corpus** 來跑 → 看它是否真的幫你找出 **intervention**、**residual**、**No-Go**、**lineage**、**distillation** → 再回頭看這套方法是怎樣從 **SMFT**／成界之學的研究歷史長出來。

這是完全不同的入口。

一、最重要的改變：由「傳播理論」變成「傳播一件可以使用的研究工具」

《The Semantic Collider》自己其實已經走到這一步：它不是把 LLM 定義為真理來源，而是把 Human Scientist、LLM、External World 分別理解為 epistemic governance、relational search、final adjudication；而且真正的 scientific artifact 可以是 **Narrative Paper + Trace Ledger**，而不是只有最後的文章。 The Semantic Collider From AI-G... The Semantic Collider From AI-G...

所以 Git repository 不是單純「方便下載 PDF」。

如果做好，它實際上是把：

paper

轉換成：

executable methodology。

這會令 SMFT／成界之學由：

> 一套要求讀者先理解的龐大理論

變成：

> 一套曾經產生某種 Human–AI research technology 的 origin case。

這個位置其實容易傳播得多。

二、你的兩個方向，我會選：**2** 為核心，**1** 為外殼

你提出：

方案 **1**

把全部文章放 Git，配一個 Skill，讓 AI 幫讀者：

- 閱讀；
- 解釋；
- 重做分析；

- 回答 baseline corpus 的問題。

這很好，而且是很實用的 **Archive + Tutor**。

但是如果只做這一步，最後可能仍然變成：

> 「一個很好的文件庫 + 一個會回答這些文件問題的 chatbot。」

它未必構成真正新的研究產品。

方案 **2**

把：

- **The Semantic Collider**
- **Reconstructable Research**
- **From Dialogue to Research Architecture**
- **Short Version**

重寫成一個真正的 **Research-Dynamics Kernel / Skill**。

然後：

《𝕆 → G₂/SO(4) → ℍ → ℂ² 成界過程初探 1–23》 = Sample Raw Dataset

這個方向的價值高很多。

因為用戶不一定需要對 SMFT 有興趣。

他甚至可以完全不懂八卦、八元數、SMFT。

他只需要說：

> 「我也有 6 個月和 AI 發展一套理論／產品／研究計劃的聊天記錄，可以幫我分析嗎？」

然後 Skill 對他的 corpus 做同一套工作。

這時你就不再只是發表：

> 一篇關於自己的 Human–AI collaboration 的文章。

而是在提供：

> 一種可以套用到其他人的 **Human–AI research history** 上的分析 **instrument**。

這個轉變才真正大。

三、最合理的整套架構，我會這樣安排

不是：

```
20 PDFs
+ README
+ one prompt
```

而是：

```
Framework Kernel
        ↓
Analysis Skill
        ↓
Raw Human–AI Corpus
        ↓
Machine-readable Research Reconstruction
        ↓
Intervention / Residual / Lineage / No-Go Analysis
        ↓
Research Distillation Analysis
        ↓
Replay / Validation Plan
```

而你的資料可以很漂亮地分成以下幾層。

Layer 0 — Framework Kernel

這裡只放方法本身。

核心來源：

A. The Semantic Collider

負責：

- Conceptual Beam
- Native Reconstruction
- Constraint
- Collision
- Residual
- Failed Mapping
- Invariant
- Lineage
- Holdout
- Independent Recurrence
- Falsification

它本身已有一個很適合變成 runtime kernel 的八步核心：

> **Reconstruct → Declare → Abstract → Collide → Break → Ledger → Transfer → Validate**。 The Semantic Collider From AI-G…

B. Reconstructable Research

負責另一個維度：

- Event
- Claim State
- Transformation
- Residual
- Evidence
- Provenance

- Genealogy
- Reconstruction Assertion
- Human Intervention
- Replay

即：

> Semantic Collider 告訴 Skill 概念之間發生什麼；
>
> Reconstructable Research 告訴 Skill 研究歷史怎樣保存及重建這件事。

C. From Dialogue to Research Architecture

這份其實是：

> 兩個 **framework** 如何落在人機長期研究上的 **integration layer**。

例如：

- Human intervention taxonomy
- search-space governance
- distillation cascade
- replay experiments
- model-initiated corrections
- intervention atlas

D. Short Version

這個不是理論文件。

我反而會把它定位為：

> **Kernel Manifest**

即 Skill 首先要掌握的 10 個最高層 invariants。

不用一開始塞幾百頁給模型。

四、《$\mathbb{O} \to G_2/SO(4) \to \mathbb{H} \to \mathbb{C}^2$ 1–23》的角色非常漂亮

我不會叫它：

> supporting document

我會正式叫：

Sample Dataset A — Long-Horizon Human–AI Theory Formation

甚至：

Natural-History Corpus A

因為《Semantic Collider》本身也很明確地說：

> motivating corpus 是 natural history，而不是 Semantic Collider hypothesis 的獨立證明；同一 research lineage 裡重複出現的概念不能當作 independent recurrence。 The Semantic Collider From AI-G…

這其實反而非常適合 Git repo。

因為：

```
RAW DATA
↓
ANALYSIS
↓
DISTILLED OUTPUT
```

三層全部都有。

這在方法學 repo 裡非常罕有。

五、《與 **ChatGPT 5.6** 探討**……**開宗立派還欠缺的準備材料》的最佳角色

這份文件我反而不建議當 **Raw Data**，也不建議放進 **Skill** 的 **blind first pass**。

它最適合成為：

Bridge / Annotation Corpus

也就是：

> Raw 1–23 與最後四份 English programme documents 之間的「研究者整理層」。

它記錄了：

- 怎樣重新分類成果；
- 哪些要進 Formal Core；
- 哪些要降為 Extension；
- 哪些變成 No-Go；
- 哪些變成 experiment；
- 哪些需要 refactoring；
- 哪些 claim 太強；
- 哪些研究缺口仍然存在。

換句話說：

```
Raw Dialogue
      ↓
Researcher / AI Meta-Analysis
      ↓
Distilled Research Architecture
```

它就是中間那一層。

六、四份英文綱領則幾乎天然是「**Gold Outputs**」

這一點我覺得特別漂亮。

四份英文文件：

1. **Research Programme**
2. **Formal Core**
3. **Experimental Programme**
4. **E4 Preregistration**

剛好就是：

```
Research Programme
        ↓
Formal Core
        ↓
Experimental Programme
        ↓
Preregistration
```

所以它們不只是 documents。

它們可以成為：

Reference / Gold Distillation Outputs

Skill 可以拿 Raw Corpus 做 analysis。

最後問：

> 從 raw corpus 本身，AI 能不能重建出類似這種 distillation trajectory？

甚至可以比較：

```
Blind Reconstruction
        vs
Historical Gold Output
```

這開始有 benchmark 的味道了。

七、但這裡有一個非常重要的技術陷阱

如果 Skill 一開始就讀：

- Semantic Collider；
- long article；
- sample analysis；
- 四份 gold output；
- 然後再分析 raw 1–23，

那它很可能只是：

> 把答案投射回 **raw corpus**。

這正是 Semantic Collider 自己警告的問題。

如果 prompt 本身一直出現：

boundary, gate, trace, residual, invariance

後來 output 又找到：

boundary → gate → trace → residual

就不能說是 independent discovery。 The Semantic Collider From AI-G...

所以你的 Git Skill 最好有 三個 **Mode**。

八、我建議 **Skill** 明確分三種 **Mode**

Mode 1 — Tutorial Mode

可以讀全部 reference material。

用途：

> 「解釋這個 case 是怎樣分析的。」

這時知道答案完全沒有問題。

Mode 2 — Audit Mode

可以使用 Semantic Collider / Reconstructable Research 的 methodology。

但：

不要先讀 **Sample Gold Analysis**。

輸入：

```
user corpus
```

輸出：

```
events
claims
residuals
interventions
lineage
No-Go candidates
distillation map
```

Mode 3 — Blind Discovery Mode

最嚴格。

Skill 甚至不應預先假定：

- Gate

- Trace
- Ledger
- Purpose
- Complex Structure
- Four Phases

一定要出現。

先做：

```
Native Reconstruction
→ Structural Extraction
→ Intervention Detection
→ Residual Detection
```

之後才和 reference framework 比較。

這樣才真正符合 Semantic Collider 的精神：

> **do not maximize similarity; preserve constraint and mismatch.**

九、**Git repository** 最重要的不是「文件」，而是這條 **runtime**

我會希望用戶可以做：

```
/analyze my_research_corpus/
```

然後得到：

```
01_research_timeline.md
02_claim_states.md
03_intervention_atlas.md
04_residual_ledger.md
05_nogo_ledger.md
06_lineage_map.md
07_distillation_map.md
08_model_corrections.md
09_human_governance.md
10_replay_candidates.md
```

再加 machine-readable：

```
research_trace.json
claims.json
residuals.json
interventions.json
lineage.json
replay_plan.json
```

這才是真正的：

> **Reconstructable Research runtime**

而不是 AI 幫忙 summarize PDF。

《Semantic Collider》本身已經提出 machine-readable Trace Ledger，內容包括 beam cards、sources、prompts、model settings、native reconstructions、candidate mappings、rejected mappings、residuals、revision history、replications 和 holdouts。 The Semantic Collider From

所以你的 Git 其實是把文章裡已經提出的東西真正做出來。

十、甚至可以將 **Skill** 理解成一個「**Compiler**」

這和你之前的 Runtime Kernel 思路很吻合。

輸入：

```
Raw Human-AI Interaction
```

Intermediate Representation：

```
Research Event Graph
Claim State
Beam
Constraint
Residual
Intervention
Lineage
Evidence
```

輸出：

```
Research Programme
Formal Core Candidates
No-Go Ledger
Experiment Candidates
Replay Plan
```

即：

Human–AI Research Corpus → Research IR → Research Architecture

這個定位很有力。

十一、這會怎樣改變 **SMFT**／成界之學的傳播？

我認為最大的改變，是傳播順序會完全反過來。

現在：

```
SMFT
↓
成界之學
↓
大量文章
↓
讀者要理解
↓
也許相信它有價值
```

新的方式：

```
Useful Research Skill
↓
User applies it to own project
↓
```

```
Gets useful reconstruction
↓
Discovers the Sample Corpus
↓
Asks where the method came from
↓
Discovers Semantic Collider / Reconstructable Research
↓
Eventually encounters SMFT / 成界之學
```

這可以叫：

Reverse Propagation

不是先傳播 ontology。

先傳播：

capability。

十二、這對 **SMFT** 特別重要

SMFT／成界之學最大的傳播障礙之一，很可能不是：

> 沒有足夠內容。

而是恰恰相反：

> 內容太多，進場成本太高。

一個外部研究者看到：

- SMFT；
- 成界之學；
- 八元數；
- 八卦；
- observer；
- quantum；
- recursive declaration；

很容易在真正看懂方法以前就停止。

但如果他看到：

> “Upload your long Human–AI research history. This Skill reconstructs hidden theory transitions, residuals, No-Go results and intervention effects.”

他根本不需要先接受 SMFT。

他可以先使用。

十三、甚至「不懂 **SMFT** 的人用了 **Skill** 覺得有用」才是更強的傳播

因為那產生的是：

```
Framework usefulness
independent of
belief in originating theory.
```

這正是你一直想要的：

> 不要靠成界之學去證明成界之學。

如果 Semantic Collider / Reconstructable Research 能分析：

- AI safety research；
- economics；
- software architecture；
- philosophy；
- medical theory formation；
- mathematics exploration；

而使用者覺得有用，那麼它本身就開始脫離 origin corpus 生存。

這反而令 origin corpus 更值得研究。

十四、而 **Git** 本身還可以解決你目前方法最重要的一個缺口

現在最弱的地方之一是：

> 全部都是同一 **lineage**。

《Semantic Collider》自己也很清楚：

Sequential collision：

$A \times B \rightarrow I_1$
$I_1 \times C \rightarrow I_2$
$I_2 \times D \rightarrow I_3$

會造成 conceptual inheritance，因此必須保留 lineage，而且 BeamReuse 會引起 attractor lock-in；所以需要 alternative vocabularies、competing abstractions、blinded evaluators、beam ablation、random controls。 The Semantic Collider From AI-G...

Git repo 有一個非常有意思的效果：

外部人可以 **fork**。

然後：

User A 用自己的 corpus。
User B 用自己的 corpus。
User C 用自己的 corpus。

結果可能：

```
Corpus A → I_A
Corpus B → I_B
Corpus C → I_C
```

如果不同人、不同模型、不同 domain：

```
BlindCompare(I_A,I_B,I_C)
```

仍出現某些相近 intervention / residual / distillation structures，

這時才開始產生真正重要的：

Independent Recurrence

而 Semantic Collider 本身就明確定義：

> 同一 conceptual lineage 的重複 elaboration 不算 independent recurrence。 The Semantic Collider From AI-G...

所以 Git 不只是 dissemination platform。

它可以變成：

> **framework** 自己的 **validation infrastructure**。

這一點我覺得特別重要。

十五、所以傳播鏈會由「讀者」變成「參與者」

舊模式：

```
Author
↓
Paper
↓
Reader
```

新的模式：

```
Framework
       ↓
Skill
       ↓
User Corpus
       ↓
User Analysis
       ↓
Fork / Issue / Result
       ↓
Cross-case Comparison
       ↓
Framework Revision
```

這就不再只是 publication。

它開始接近：

Open Research Instrument

十六、我甚至不會把 **Repo** 的首頁主題寫成 **SMFT**

如果目標是最大化傳播，我會讓 repo 的入口是：

Human–AI Research Dynamics Toolkit

或者：

Semantic Collider + Reconstructable Research Toolkit

副標題：

> *Turn long Human–AI research histories into auditable intervention, residual, lineage, distillation and replay maps.*

然後：

> **Sample Case: The $\mathbb{O} \to G_2/SO(4) \to \mathbb{H} \to \mathbb{C}^2$ World-Formation Research Programme**

SMFT／成界之學作為 **origin case**。

這反而比：

> SMFT Research Repository

更容易讓陌生人進場。

十七、我會把 **Repository** 設計成這樣

```
/
├── README.md
│
├── kernel/
│   ├── semantic-collider.md
│   ├── reconstructable-research.md
│   ├── research-dynamics.md
│   └── kernel-short.md
│
├── skill/
│   ├── SKILL.md
│   ├── coding-rules.md
│   ├── output-schema.md
│   └── modes/
│       ├── tutorial.md
│       ├── audit.md
│       └── blind-discovery.md
│
├── sample/
│   └── world-formation-case/
│       ├── raw/
│       │   └── O-G2-H-C2_1-23.pdf
│       │
│       ├── bridge/
│       │   └── missing-materials-analysis.pdf
│       │
│       ├── gold/
│       │   ├── research-programme.pdf
│       │   ├── formal-core.pdf
│       │   ├── experimental-programme.pdf
│       │   └── E4-preregistration.pdf
```

```
│       │
│       └── reference-analysis/
│           ├── intervention-atlas.md
│           ├── residual-ledger.md
│           ├── nogo-ledger.md
│           └── distillation-map.md
│
├── schema/
│   ├── event.schema.json
│   ├── claim.schema.json
│   ├── residual.schema.json
│   ├── intervention.schema.json
│   └── lineage.schema.json
│
├── examples/
│   └── minimal-demo/
│
├── benchmark/
│   ├── blind-analysis/
│   ├── anti-attractor-controls/
│   └── replay/
│
└── docs/
    ├── quickstart.md
    ├── methodology.md
    └── faq.md
```

這已經不是文章 archive。

是：

> **research methodology package**。

十八、最重要的一條設計原則：**Gold** 不可以污染 **Blind Analysis**

這個 repository 如果要嚴肅，我甚至會在 README 明確寫：

```
DO NOT load /gold or /reference-analysis
during Blind Discovery Mode.
```

流程：

```
RAW
↓
blind analysis
↓
freeze output
↓
open GOLD
↓
compare
```

而不是：

```
RAW + GOLD
↓
"discover" GOLD
```

這會令 repo 的可信度提升很多。

也非常符合《Semantic Collider》：

> protocol before outcome；否則研究者可以選 beam、透露 hints、反覆 reprompt、隱藏 failed mappings，造成嚴重 selection bias。 The Semantic Collider From AI-G...

十九、我覺得還可以多加一個很重要的功能

不是只有：

> Analyze my corpus.

還可以有：

Compare my corpus with the sample case

但比較的不是：

> 你的 theory 和 SMFT 有沒有一樣。

而是：

```
research dynamics comparison
```

例如：

Dimension	Your corpus	World-Formation case
Beam additions		
Residual survival		
Model corrections		
Human reframing		
No-Go formation		
Theory downgrades		
Distillation depth		
Preregistration		
Lineage control		
Replay readiness		

這樣讀者會自然地和你的 case 發生關係。

但又不需要接受你的 physics / ontology。

二十、這甚至可能比發表更多 **SMFT** 文章更有效

因為再增加一篇：

> 200 頁 SMFT paper

對傳播的 marginal gain 很可能已經很低。

但一個可以使用的：

> **Human–AI Research Dynamics Skill**

可能開一條全新的 audience。

尤其是：

- AI researchers；
- prompt / agent researchers；
- philosophy-of-science people；
- AI-assisted mathematicians；
- independent researchers；
- developers using long-running AI Projects；
- people with months of Claude / ChatGPT research histories。

他們未必會搜尋：

> SMFT

但會搜尋：

> “How do I analyze a year of AI-assisted research conversations?”

這就是新的入口。

二十一、不過我會把「根本性改變」分成三個層次

層面	我的判斷
SMFT／成界之學的大眾曝光	有幫助，但未必立即根本改變
高質量技術讀者進入框架的門檻	可能根本改變
Semantic Collider / Reconstructable Research 成為獨立方法的可能性	很大程度改變
SMFT 的科學可信度	不會因 Git 自動提升
未來獨立 **validation** 的可能性	明顯提升

所以我會很精確地說：

> 它不會直接把 **SMFT** 從 **speculative programme** 變成 **validated theory**；但它有可能把 **SMFT**／成界之學從一套「需要別人先讀懂的私人研究體系」，轉化成一個「別人可以進來操作、**fork**、驗證、反駁、重建的研究生態」。

這才是根本性的地方。

二十二、而且這其實非常符合《**Semantic Collider**》原來最深的一句話

它把完整研究物件理解成：

> **ScientificArtifact = NarrativePaper + TraceLedger**。 The Semantic Collider From AI-G...

你現在其實可以再往前一步：

> **ScientificArtifact = Framework + Kernel + Raw Trace + Gold Distillation**

+ Replay Protocol

也就是：

```
Paper
        ↓
Executable Paper
        ↓
Executable Research Method
```

這個演化，我認為比單純把文章縮短更重要。

我目前最建議的決策

不要在方案 **1** 和方案 **2** 之間二選一。

做成：

> 方案 **2** 是產品，方案 **1** 是 **sample/documentation layer**。

即：

```
MAIN PRODUCT
Semantic Collider
+ Reconstructable Research
+ Research Dynamics Kernel
+ Skill

REFERENCE CASE
𝕆 → G₂/SO(4) → ℍ → ℂ² 1–23

ANNOTATION / BRIDGE
開宗立派還欠缺的準備材料

GOLD DISTILLATION
Research Programme
→ Formal Core
→ Experimental Programme
→ E4 Preregistration
```

如果這樣做，我會認為它不只是幫助宣傳 **SMFT**／成界之學。

它是在改變別人接觸 **SMFT**／成界之學的方式：

> 從「先理解我的理論」變成「先使用由這套理論研究歷史產生的方法」。

對你目前「文章太長、人類讀者生態難以傳播」這個問題，這是比再寫一篇更短文章更根本的解法。

Shape the repository into a usable research instrument

- Draft the repository README
- Define the three analysis modes

我最欣賞的，是若成功傳播這套工具，很多人便被動地要粗略看看那份 **Sample** 文件。這樣，他們可能會把那份 **Sample** 文件，介紹給有做相關研究的人員。

對，我覺得你抓到這個方案一個比「宣傳工具」更有價值的二階傳播機制。

不是要求使用者主動對 SMFT、成界之學產生興趣，而是：

> 因為要學會／驗證這套工具，他不得不粗略接觸 **Sample**；而 **Sample** 裏面若碰巧出現他認識的研究方向，他就可能把它轉介給真正懂那個領域的人。

這和直接宣傳理論的傳播路徑完全不同。

傳統路徑

看見 SMFT／成界之學
→ 願意讀
→ 覺得有興趣
→ 再介紹別人

第一步門檻已經很高。

你現在這條路徑

想用 Human-AI Research Dynamics Tool
→ 看 Sample 怎樣分析
→ 被動掃描 $\mathbb{O} \to G_2/SO(4) \to \mathbb{H} \to \mathbb{C}^2$ corpus
→ 發現其中某段和自己的專業／朋友的研究有關
→ 「這部分你應該看看」
→ 專業人士接觸原始理論

這其實是很不同的 **referral topology**。

特別有利的是：不同人會看到不同的東西

一個 AI researcher 可能只注意：

- attractor formation；
- observer；
- long-horizon reasoning；
- Purpose architecture；
- intervention replay。

一個 mathematician 可能只注意：

- $\mathbb{O}$；
- $G_2/SO(4)$；
- quaternionic subalgebras；
- $\mathbb{C}^2$ polarization。

一個 foundations-of-physics researcher 可能注意：

- observer;
- declaration;
- filtration;
- pre-time;
- measurement.

一個研究《易》的讀者則可能注意：

- 先天／後天；
- 四象；

- 八卦；
- 九宮；
- 成界過程。

所以 Sample 不需要令每一個人理解整套理論。

它只需要產生：

> 「這一小段，好像應該給某某人看看。」

這種傳播其實可能比「我自己完全讀懂並認同後才分享」容易很多。

而且 **Sample** 的長度在這裏反而未必全是缺點

作為文章，950 頁當然幾乎不可讀。

但作為 **reference corpus / sample raw data**，長反而有另一種價值：

> 它是一個可以被 AI 搜尋、抽取、導航，而不是要求人類線性閱讀的「研究礦床」。

使用者可能只問：

> “Show me the episodes related to octonions.”

或：

> “Why was the Four Symbols interpretation downgraded?”

或：

> “Which parts concern LLM sudden understanding?”

他不需要讀 950 頁。

但每一次 retrieval 都可能把他帶到一個原本不知道存在的研究分支。

所以 Sample 的角色最好不是：

Read this book.

而是：

Explore this research trace.

這個定位差很多。

我甚至會故意設計一個 `sample-topic-map.md`

例如：

If you work on…	Look at these sample threads
LLM interpretability	attractors, sudden understanding, phase transition
AI agents	Purpose, observer memory, revision, latching
Quantum foundations	self-referential observer, declaration, filtration

If you work on…	Look at these sample threads
Octonions / exceptional geometry	$\mathbb{O} \to G_2/SO(4) \to \mathbb{H}$
Complex structures	$\mathbb{H} \to \mathbb{C}^2$, polarization, $J^2 = -I$
Philosophy of science	residuals, No-Go, blind derivation
I Ching studies	Xiantian/Houtian, Four Symbols, trigrams
Scientific methodology	Semantic Collider, reconstruction, replay

這不是替 Sample 做結論。

它只是提供：

> 可能值得轉介給哪類人的入口。

還可以加入一個非常簡單的 **Skill command**

例如：

```
/find-relevance <research field>
```

使用者輸入：

```
/find-relevance mechanistic interpretability
```

Skill 回答：

```
This sample corpus contains 4 potentially relevant research threads:
1. Strong Attractor → Insight Attractor
2. sudden capability transition
3. observer-dependent coarse graining
4. phase-transition interpretation of learned structure

These are exploratory correspondences, not validated results.
Recommended source episodes: ...
```

然後一鍵得到：

> **“Share this with someone working on mechanistic interpretability.”**

這就把你剛剛描述的偶然 referral，變成一個有意識設計的傳播功能。

最有意思的是：這種傳播並不要求對方支持 **SMFT**

甚至相反。

一個專家可能收到朋友轉介後說：

> 「整套理論我不接受，但其中 $G_2/SO(4) \to \mathbb{H}$ 的問題很有意思。」

這已經是成功。

另一個可能說：

> 「Purpose Belt 我不認同，但你這個 ablation 設計值得測。」

也是成功。

再有人可能說：

> 「八卦部分我完全沒有興趣，但 Human–AI intervention trace 很有研究價值。」

仍然是成功。

因為傳播單位由：

whole theory

變成：

researchable fragment。

這甚至與 **Semantic Collider** 的精神完全一致

一個成熟 conceptual beam 不需要整套 ontology 被接受。

只需要它有足夠內部結構，讓某一部分可以：

- 被獨立重建；
- 被約束；
- 被碰撞；
- 被反駁；
- 被轉移。

而 Semantic Collider 本身正強調：好的 collision 不是把不同領域全部說成一樣，而是保留 surviving structure、residual 和 failed mapping。 The Semantic Collider From AI-G...

所以對 SMFT／成界之學的最佳傳播，未必是：

> 「請接受整套 worldview。」

反而可能是：

> 「這裏有很多可以獨立拆出來檢查的 research beams。」

我會把這個效應叫作：

Tool-Mediated Incidental Exposure

工具導致使用者被動接觸 origin corpus。

然後再產生：

Expert Referral Propagation

使用者把其中與某專業相關的 fragment 轉介給真正的 domain expert。

完整鏈條是：

```
Useful Tool
→ Sample Inspection
→ Incidental Theory Exposure
→ Local Relevance Detection
```

```
→ Expert Referral
→ Specialist Evaluation
→ Possible Independent Research
```

如果真的發生，這比「多幾百個人點進文章」有價值得多。

因為你真正需要的未必是大量普通 readers。

而是少數：

> 「我不研究整套 **SMFT**，但其中這個問題正是我的專業。」

的人。

所以我現在甚至更支持這個 Git / Skill 計劃：它最有潛力的傳播機制，可能根本不是把使用者變成 SMFT 讀者，而是把使用者變成 **SMFT** 各局部問題的轉介節點。

不過我其實有點模糊。上述這個 **Kernel Skill** 我發明出來，其實是幫助 用戶 解答/解決 什麼問題的呢？我想過的可能有：

- 單純分析討論的結構，然後抽出一些隱藏的結果，或可以發展的方向。
- 點出討論過程中的錯誤、漏洞、可改善、隱藏重大變化可能、在新突破的門口。。。等等的關鍵節點。
- 用另一種眼光，匯報給高層這篇討論的特殊性。結果、發展方向、漏洞、關鍵節點、新突破的門口。。。
- 建議如何修改關鍵討論節點，讓用戶重新再討論。
- 提出看似天外飛仙的新奇變化，
- 提供一幅非常龐大的對撞關係分析圖譜
- 。。。很多其它可能**1**

你現在的模糊其實很正常，因為你列出的東西混合了「核心問題」、「分析能力」、「輸出形式」和「進階玩法」。

我會把它收斂成一句：

> 這個 **Kernel Skill** 的核心用途，是把一段長期、非線性、混亂的人**–AI** 討論，轉換成「可重建的研究狀態變化圖」，然後指出：究竟發生了什麼、哪裏重要、哪裏有問題、哪裏值得重開，以及下一步最值得試什麼。

這樣就清楚很多。

它真正解答的是 **5** 個核心問題

1. What actually happened?

不是摘要內容，而是重建：

- 哪個想法何時出現；

- 哪個被修正；
- 哪個被降級；
- 哪個被放棄；
- 哪個 residual 一直未解；
- 哪個 later idea 是從哪裏長出來的。

即：

> **Research Trace Reconstruction**

---

2. What really mattered?

一千輪討論中，真正改變後續方向的可能只有十幾個節點。

Kernel 要找：

- 關鍵轉折；
- 隱藏重大修改；
- model correction；
- human reframe；
- No-Go formation；
- branch split；
- theory compression；
- breakthrough threshold。

這正是你說的：

> 「漏洞、重大變化、新突破門口、關鍵節點」

可以叫：

> **Critical Transition Detection**

---

3. What is wrong or still unresolved?

很多對話表面已經「回答了」，實際只是語言變漂亮。

Kernel 要找：

- 未證明的跳步；
- framework elasticity；
- 偷換定義；
- 過度統一；
- lineage contamination；
- 被掩蓋的 residual；
- 只 compatible 但不 necessary；
- 被過早關閉的分支。

即：

> **Residual / Failure / No-Go Audit**

這可能是整套工具最有價值的功能之一。

---

4. What should we revisit or change?

這不是只分析歷史，而是問：

> 「如果這一個節點當時換一個處理方法，研究可能會去哪？」

例如：

- 這裡不要接受 model 的回答；
- 這裡應加 blind derivation；
- 這裡應加入另一個 beam；
- 這裡應做 sham control；
- 這裡應把 claim downgrade；
- 這裡應重開 branch。

這就是：

> **Counterfactual Research Intervention**

你之前說：

> 「建議如何修改關鍵討論節點，讓用戶重新再討論」

這其實非常核心。

---

5. What is the best next search?

最後才是：

- 下一條理論線；
- 下一個 collision；
- 下一個 experiment；
- 下一個 document；
- 下一個 replay；
- 下一個 specialist referral。

即：

> **Research Opportunity Generation**

所以真正的產品，不是「分析聊天」

而是：

Research Dynamics Analyzer

或者更完整：

Human–AI Research Dynamics Kernel

它處理的問題是：

> 我和 **AI** 已經談了很多很多東西，但我不知道這一大堆東西裏面真正發生了什麼、哪些轉折最重要、哪些錯誤未處理、哪些成果被埋沒，以及現在最值得往哪裏走。

這就是一個非常實在的 user pain。

你列出的功能，可以重新分類

你想到的功能	真正角色
分析討論結構	Core
抽出隱藏結果	Core
找錯誤、漏洞	Core
找重大變化	Core
找突破門口	Core
建議重開哪些節點	Core
幫高層匯報	Projection / Output Mode
提出天外飛仙新方向	Exploration Extension
畫巨大對撞關係圖	Visualization / Representation
找可以發展的方向	Core
比較不同 branch	Core
找誰影響了誰	Core
建議 replay / ablation	Advanced Core
判斷哪些 claim 應升降級	Core

所以其實不是「很多不相關功能」。

它們可以收斂在同一個核心：

> 理解研究狀態如何變化，並利用這個理解改善下一步研究。

我會把 **Kernel** 定義成 **4** 個主模組

A. RECONSTRUCT — 重建

回答：

> 到底發生過什麼？

輸出：

- timeline
- claim states
- branch tree
- lineage
- model / human intervention map

B. DIAGNOSE — 診斷

回答：

哪裏有問題？

輸出：

- residual ledger
- failed mappings
- No-Go candidates
- overclaim
- hidden assumption
- premature closure
- attractor lock-in

C. DISCOVER — 發現

回答：

有什麼其實已經埋在裏面，但當時沒看見？

輸出：

- hidden invariant
- underdeveloped branch
- latent hypothesis
- unexplored collision
- specialist-relevant fragment
- possible breakthrough frontier

D. INTERVENE — 干預

回答：

現在應該怎樣重新走？

輸出：

- reopen node
- add beam
- remove beam
- downgrade claim
- blind derivation
- replay experiment
- sham control
- next prompt
- next branch
- next experiment

整套就是：

Reconstruct → Diagnose → Discover → Intervene

這個我覺得可以成為整個 Skill 的最簡單核心。

「高層匯報」其實不是第五個核心

這個很重要。

你說：

> 用另一種眼光，匯報給高層這篇討論的特殊性。

這不是不同的分析。

而是同一個 Research State 的另一個 **Projection**。

例如同一批資料可以輸出：

Researcher View

詳細 residual / lineage / claims。

Executive View

只看：

- 3 個最大突破；
- 3 個最大風險；
- 3 個最值得投資方向；
- 5 個 decisive turning points。

Reviewer View

只看：

- unsupported claims；
- missing controls；
- falsification routes。

Historian View

只看：

- intellectual genealogy；
- branch mutation；
- human vs AI roles。

所以：

> **Analysis Core** 一個，**Projection** 可以很多。

這會令 Kernel 設計乾淨很多。

「天外飛仙」也不應該放進 **Core**

這個功能很吸引，但危險。

因為它會令：

> forensic analysis

和：

> speculative ideation

混在一起。

最好變成一個可開關模式：

`Exploration Mode`

在完成 reconstruction 後才允許：

- distant beam proposal；
- strange analogy；
- unexpected mathematics；
- cross-domain collision；
- speculative next world。

即：

> 先知道發生了什麼，再容許 **AI** 發瘋。

而不是一開始就發瘋。

巨大對撞圖譜也是 **Output**，不是 **Purpose**

圖譜是非常有價值的，但它是 representation。

底層其實是：

```
Node:
  Claim
  Residual
  Beam
  Constraint
  Intervention
  Evidence
  No-Go

Edge:
  caused-by
  revised-by
  contradicts
  derives-from
  motivates
  blocks
  reopens
  generalizes
```

圖只是這個 graph 的可視化。

所以 Kernel 的核心不是：

> 畫 graph

而是：

> 建立可信的 **Research Event Graph**

最值得打中的使用場景，其實非常具體

例如一個人用了 ChatGPT / Claude 做了半年研究，有：

- 300 個 chats；
- 20 份 draft；
- 50 個 ideas；
- 很多重複；
- 自己已忘記哪些地方重要。

他真正的問題不是：

> 「幫我 summarize。」

而是：

> 「我到底已經研究到哪裏？」

再下一層：

> 「有什麼其實已經突破了，但我沒有注意？」

再下一層：

> 「有哪些漂亮結論其實是錯的？」

再下一層：

> 「如果重新跑一次，我應在哪幾個節點改變討論方式？」

這就是你的產品。

我甚至會給它一句非常清楚的首頁文案

Turn Long Human–AI Conversations into Research Intelligence

副標題：

> **Reconstruct what changed, detect what failed, surface what was missed, and identify where to intervene next.**

這比：

> Semantic Collider analysis tool

更容易讓陌生人立即明白。

然後 Semantic Collider / Reconstructable Research 才是 underneath 的 kernel。

還可以進一步變成 **6** 個標準問題

每個 corpus 都跑同一套：

1. **What changed?**
2. **What caused the change?**
3. **What failed or remains unresolved?**
4. **What was overlooked?**
5. **Which nodes are worth replaying?**
6. **What should happen next?**

如果這六題能答得好，這個 Skill 已經非常有用。

我認為最核心的 **breakthrough** 是這一點

普通 AI 讀 conversation：

> **understands content**

你的 Kernel 要做：

> **understands research evolution**

這兩個不是同一件事。

普通 summarize 是：

```
Discussion
→ Topics
→ Conclusions
```

你的 Kernel 是：

```
Discussion
→ Research States
→ Transitions
→ Residuals
→ Interventions
→ Counterfactuals
→ Next Search
```

這就是它真正的 product identity。

所以，如果現在一定要我替它下一個最精準定義，我會寫：

> **A Research Dynamics Kernel for reconstructing, diagnosing, and redirecting long-horizon Human–AI intellectual work.**

再白話一點：

> 它不是幫你重讀聊天，而是幫你看見「研究本身是怎樣變化的」。

這個 **Skill** 若放入 **OpenAI** 網頁版做，直接寫要求（不上細節）也大概可以執行了。
但對於 **Codex** 或 **Claude Code** 的設置，特別是用了第三方 **LLM** 的情況，即使有 **1M Token**，**Codex** 或 **Claude Code** 處理超額 **Token** 的能力，未必可以完

善整理質料，輸出理想的分析結果？

對。而且你這個 **Skill** 的任務，恰恰是最不應該迷信「**1M context =** 全部塞進去一次分析」的類型。

原因不是 Codex 或 Claude Code 不夠強，而是你要找的不是普通摘要，而是：

> 一個早期看似不起眼的句子，可能到 **200** 頁之後才突然成為重大轉折。

這種資訊最容易被 compaction、summary 或一次性超長 context 淹沒。

目前 Codex 本身會在接近 context 上限時自動做 compaction；OpenAI 的 Responses API 也正式提供 compaction，目的是把較早內容壓縮後繼續長任務。[OpenAI Developers] Claude Code 也採用類似策略：Anthropic 說明它會把歷史交給模型做摘要壓縮，保留重要決定、未解 bug 等，再以壓縮內容和最近使用的檔案繼續；Anthropic 同時明確指出，過度 compaction 可能丟掉那些「當時看似不重要、後來才發現關鍵」的細節。[Anthropic]

而這正是我們這個 Skill 最怕的東西。

---

1. 1M Token 解決的是「容量」，不是「研究重建」

假設真的有：

1,000,000-token context

也不等於：

1,000,000-token perfect research understanding.

至少有五個不同問題。

A. Attention dilution

重要資訊只佔整個 corpus 的極小部分。

例如一句：

> 「不對，四象不應該是 ontology。」

可能只佔 20 tokens。

但它卻可能改變後面 100,000 tokens。

如果只是把 1M tokens 一次放進去要求：

> “Analyze the whole research process.”

模型很可能知道這句存在，但未必知道它是 **trajectory-changing event**。

B. Local importance ≠ global importance

某個節點在當下看似普通：

```
Episode 7:
Maybe this distinction should not be fundamental.
```

到了 Episode 19 才發現：

```
Episode 7
→ removed one assumption
→ enabled blind derivation
→ produced No-Go result
→ changed experimental design
```

所以需要的是：

retrospective dependency reconstruction

而不是閱讀時即時摘要。

C. Compaction會天然偏向「現在看來重要的內容」

一般 compaction 很合理地保存：

- current task；
- current architecture；
- decisions；
- unresolved items。

但你的研究分析反而需要保存：

- rejected old claim；
- seemingly irrelevant objection；
- abandoned branch；
- temporary misunderstanding；
- failed analogy。

因為那些東西後來可能是：

> Critical Intervention Atlas 的核心資料。

所以對一般 coding agent 而言是「可以丟掉的垃圾」，

對 Research Dynamics Kernel 而言可能是：

> 研究證據。

D. Lost-in-the-middle / salience problem

即使全部仍在 window 裏，模型也不代表會同等利用所有位置。

超長 context 最適合：

> 「我需要某資料時，它有可能仍然在那裡。」

而不是保證：

> 「模型會自動建立整個一百萬 token 的正確因果圖。」

這兩件事差很遠。

E. 一次分析沒有 **auditability**

如果模型讀了 1M tokens 然後說：

> 「我發現 11 個 critical turning points。」

你會問：

> 為什麼是這 11 個？

如果沒有 intermediate records，就很難檢查：

- 漏了什麼；
- 哪裏誤判；
- 哪些 node 是後見之明；
- 哪些關係是模型自己幻想出來。

---

2. 所以 **Codex / Claude Code** 最好不要被設計成「一次讀完」

我反而會故意設計這個 Skill：

> 即使只有 **64K–128K** 穩定 **context**，也可以可靠運行。

1M context 只是 bonus。

這會令 Skill 對：

- Codex；
- Claude Code；
- Ollama；
- Qwen；
- DeepSeek；
- GLM；
- 其他 OpenAI-compatible models

都更 robust。

---

3. 尤其第三方 **LLM**，問題會更明顯

這點你的判斷很準。

Codex harness 對 OpenAI 自己的 model 有特定的 agent-loop、prompt、tool-use 和 compaction 配合。OpenAI 官方也明確提醒：若原來的 harness/prompt 是為其他模型或第三方模型設計，換模型時往往需要更大的 prompt/tool 調整，而不是假設完全可互換。[OpenAI Developers]

同樣地，Claude Code 的 context-management 行為是和 Claude 的長程狀態管理及 compaction 配合設計的。[Anthropic]

若你把下面的東西換成第三方：

```
Claude Code
      ↓
OpenAI-compatible proxy
```

```
      ↓
Qwen / DeepSeek / local LLM
```

表面 API 可能完全正常。

但以下未必相等：

```
tool discipline
summary quality
salience judgement
context awareness
compaction fidelity
long-horizon state tracking
```

所以：

> **API compatibility ≠ agentic cognitive compatibility.**

---

4. 這反而告訴我們 **Kernel** 應該怎樣設計

不要：

```
LOAD EVERYTHING
↓
THINK VERY HARD
↓
WRITE FINAL REPORT
```

而應該：

```
RAW CORPUS
      ↓
PASS 1 — Structural Extraction
      ↓
Research IR
      ↓
PASS 2 — Cross-Section Reconstruction
      ↓
Research Graph
      ↓
PASS 3 — Diagnostic Analysis
      ↓
PASS 4 — Targeted Re-reading
      ↓
PASS 5 — Final Synthesis
```

這很重要。

---

5. **Pass 1**：先把 **raw corpus** 拆成不容易遺失的「原子」

例如每 20–50k tokens 一個 segment。

每個 segment 不要求「理解整個研究」。

只抽：

```
events
claims
questions
```

```
objections
human interventions
model corrections
residuals
branch starts
branch closes
source additions
explicit decisions
```

輸出 JSON：

```
event_0174:
  actor: human
  type: possible_reframe
  claim_before: ...
  intervention: ...
  immediate_effect: ...
  source_location: ...
  confidence: 0.82
```

最重要的是：

> **source location** 一定保留。

---

6. Pass 2：不是再讀 raw，而是讀 Research IR

例如原 corpus：

800,000 tokens

第一次 extraction 後可能只剩：

80,000 tokens structured IR

這時模型才開始問：

> 哪些 event 跨 segment 有關？

例如：

```
E37
↓
R12 opened
↓
E104 revisits R12
↓
H18 reframes it
↓
C56 replaces C31
↓
NG7 created
```

這時才形成：

Research Event Graph

---

7. Pass 3：不同專家 Pass 分開做

而不是叫同一個 prompt 做所有事情。

例如：

Pass A — Intervention Detector

只找：

```
BeamAdd
ResidualFlag
ConstraintAdd
Reframe
Downgrade
Reject
BranchSelect
MethodChange
NoGoCommit
Refactor
Commit
```

Pass B — Residual Auditor

只問：

> 哪些問題一直沒有真正解決？

Pass C — Claim Genealogist

只做：

```
Claim A
→ revised into B
→ split into C,D
→ C rejected
→ D survives
```

Pass D — Error / Overclaim Auditor

找：

- unjustified jump；
- circularity；
- overfitting；
- false equivalence；
- hidden assumption。

Pass E — Breakthrough Frontier Detector

問：

> 哪些 residual 已經累積到只差一個新 beam？

這樣會比：

> “Analyze everything deeply.”

可靠得多。

---

8. 然後做一件非常關鍵的事：Targeted Re-read

假設 Pass 3 判斷：

> Episode 8 → Episode 17 可能是重大轉折。

不要直接相信 IR。

重新回 raw corpus 拉：

```
Episode 7
Episode 8
Episode 9

Episode 16
Episode 17
Episode 18
```

重新讀原文。

然後驗證：

```
Was the claimed transition really there?
```

這就是：

> **IR generates hypotheses; raw corpus adjudicates them.**

非常像你自己整套方法。

---

9. 所以 context window 應該被視為「working memory」

而不是：

> database。

真正的 database 應該在 disk：

```
/raw/
/events/
/claims/
/residuals/
/lineage/
/interventions/
/evidence/
```

模型每次只 load：

> 目前需要的研究狀態。

這一點極其重要。

可以寫成：

> **Context = Working Memory**
> **Files = Long-Term Research Memory**

---

10. 這會令 **Codex / Claude Code** 反而非常適合

因為它們最強的地方不是：

> 有很長的聊天框。

而是：

> 可以讀寫 **filesystem**。

所以 Skill 可以自己留下：

```
.state/
├── corpus_manifest.json
├── segment_index.json
├── event_ledger.jsonl
├── claim_ledger.jsonl
├── residual_ledger.jsonl
├── intervention_ledger.jsonl
├── nogo_ledger.jsonl
├── lineage_graph.json
└── audit_log.jsonl
```

每一輪 context 被 compact 掉都沒所謂。

因為重要研究狀態已經 externalized。

---

11. 這甚至非常符合 **Reconstructable Research**

你會發現一件很漂亮的事情：

Skill 本身也應該按照 **Reconstructable Research** 的原則運作。

即不是：

> 模型腦內記住 analysis。

而是：

```
Analyze
→ Externalize
→ Ledger
→ Reload
→ Reconstruct
→ Audit
```

所以這個 Skill 的 architecture 本身，就是理論的一個 demonstration。

---

12. 我甚至會禁止 **Skill** 做「**rolling summary**」

普通 coding agent 很喜歡：

```
summary.md
```

不斷覆蓋。

對 coding 很好。

對你的任務很危險。

因為：

```
summary₁
→ summary₂(summary₁)
→ summary₃(summary₂)
→ summary₄(summary₃)
```

最後就變成：

> **summary of summary of summary**

細節會不可逆地蒸發。

---

13. 你需要的應該是 append-only ledgers

例如：

```
events.jsonl
```

永遠只 add：

```
E001
E002
E003
...
```

不覆蓋歷史。

然後 claim 也可以：

```
C17 status=active
C17 status=downgraded
C17 status=rejected
```

而不是把 C17 刪掉。

這就是：

> event sourcing

的思路。

非常適合 Reconstructable Research。

---

14. 甚至可以定一個「兩層記憶」

Layer A — Immutable Evidence

Raw corpus + extracted event snippets。

不可修改。

```
Evidence Layer
```

Layer B — Revisable Interpretation

例如：

```
E117 is probably a Reframe
confidence=0.72
```

日後可以改成：

```
E117 = ResidualFlag + Reframe
confidence=0.91
```

即：

> **Evidence immutable; interpretation revisable.**

這會非常漂亮。

---

15. 這也解決第三方小模型的問題

假設使用：

Qwen 32B
甚至 14B。

它可能不能一次理解 900 頁。

沒問題。

它可以先做：

```
local extraction
```

然後用較強 model 做：

```
global reconstruction
```

甚至：

```
cheap model:
event extraction

mid model:
claim lineage

strong model:
breakthrough analysis

human:
critical validation
```

所以 Skill 不需要綁死一個模型。

---

16. 這會形成 **Model Tiering**

例如：

Stage	Model Requirement
Segmentation	trivial / code

Stage	Model Requirement
Event extraction	cheap LLM
Claim extraction	cheap–medium
Cross-segment lineage	medium
Residual analysis	medium–strong
Critical-transition detection	strong
Counterfactual intervention	strongest
Executive synthesis	strong

這可能比全部用最貴 1M model 更好。

---

17. 尤其「天外飛仙」必須最後才做

否則小模型／第三方模型非常容易 hallucinate cross-domain relation。

所以：

```
Phase 1
Forensic Reconstruction

Phase 2
Diagnostic Analysis

Phase 3
Evidence Check

Phase 4
ONLY THEN
Speculative Collision
```

必須把：

> **what happened**

和：

> **what might happen**

分開。

---

18. 我會甚至要求 **Skill** 有 **checkpoint**

例如：

```
CHECKPOINT 1
Corpus indexed

CHECKPOINT 2
Event ledger complete

CHECKPOINT 3
Claim graph assembled

CHECKPOINT 4
Residual audit complete
```

```
CHECKPOINT 5
Critical nodes verified against raw

CHECKPOINT 6
Final analysis
```

如果 agent 中途 context 爆了：

重新開 context：

```
Read CHECKPOINT.md
Read current ledgers
Continue.
```

完全不用依賴 conversation memory。

---

19. 對你的 **Sample 1–23**，這種設計尤其合適

因為那份 corpus 本身不是普通文章。

它有：

```
early hypothesis
→ correction
→ later reuse
→ reframing
→ mathematical derivation
→ negative result
→ new conceptual beam
→ meta-methodological change
```

如果一次塞進 1M context：

模型有機會看懂很多。

但如果目標是可重複地找出同樣重要的 **nodes**，

externalized multi-pass pipeline 一定比較適合。

---

20. 因此我甚至會把「超長 **context**」降級成 **optional acceleration**

Skill README 可以寫：

> Large-context models are helpful but not required.

甚至：

> **Do not rely on the model context window as the canonical research memory.**

核心原則：

```
Context windows are temporary.
Research state is persistent.
```

---

21. Codex 現在的 native compaction 的確已經改善不少，但仍不應取代 ledger

OpenAI 現時 Codex 會在超過 `auto_compact_limit` 時自動進行 compaction；Responses API 的 compaction 也會攜帶壓縮後的 prior state，使長任務可以跨 context 延續。[OpenAI Developers]

這對 coding agent 很有幫助。

但甚至 OpenAI Support 在談 context management 時也提醒，沒有保證 lossless recall。[OpenAI Developer Community]

你的用途要求的恰恰不是：

> “大致記住我們在做什麼。”

而是：

> “不要漏掉 Episode 6 那句當時不起眼、後來成為 Episode 19 根源的 objection。”

所以仍然不同。

---

22. 因此我現在反而看到這個 Skill 的一個很清楚的技術 identity

它不是：

Mega-Context Analyzer

而是：

Research-State Compiler

輸入：

```
messy long-horizon corpus
```

第一階段：

```
Corpus
→ Event IR
```

第二階段：

```
Event IR
→ Research State Graph
```

第三階段：

```
Research State Graph
→ Diagnostics
```

第四階段：

```
Diagnostics
→ Intervention Opportunities
```

第五階段：

```
Verified Evidence
→ Final Projection
```

簡寫：

> **Corpus → IR → Graph → Diagnosis → Intervention**

這個架構甚至比我們前面說的：

> Reconstruct → Diagnose → Discover → Intervene

再技術化一層。

---

23. 而這也會成為它相對 **ChatGPT** 網頁版的一個真正優勢

你說得對：

現在 ChatGPT 網頁版，給它 Projects files，再說：

> 幫我找這批對話的重大轉折。

已經可以做得不錯。

所以如果 Git Skill 只是：

```
SKILL.md:
Please analyze deeply...
```

沒有很大價值。

真正令 **Git Skill** 值得存在的，是它建立 **deterministic-ish research pipeline**：

```
same input
→ known passes
→ known ledgers
→ source pointers
→ audit
→ known output classes
```

它不是靠：

> 「某次 GPT 剛好很聰明。」

而是把聰明變成一套較可重複的程序。

所以答案很明確

是的，你的擔心是對的。

如果把 Codex / Claude Code + 1M-context model 當作：

> 「超大腦袋，把所有東西塞進去。」

結果不一定理想，第三方模型尤其如此。

但如果我們反過來把這個限制當成設計條件：

> 模型永遠只能看到有限 **working set**；所有重要研究狀態必須 **externalize**。

那麼這個限制反而會逼出一套更像真正 **Reconstructable Research** 的 Skill architecture。

而我現在甚至認為：

> 這應該成為第一版 **Kernel** 的核心工程原則，而不是後來才做的 **optimization**。

因為這樣做出來的工具，才真的不只是「一條很厲害的 prompt」。

我打算分兩個層次做。第一層次，是假設用戶是用類似 **OpenAI GPT 5.6 Sol** 網頁版 的强大 **LLM**。只要定義用戶需要把什麼內容打包到一個 **tar** 檔案。然後假設該 **LLM** 有 類似 **Project** 的設置，接受 一些基本答案，例如 **Skills in kernel format** 和 其它。主 **Prompt** 可以是 **Project file** 或 **Project System Prompt**。這個 **Skill** 大概需要 **LLM output** 幾輪分析圖譜 作爲中間產物，然後輸出你上述設計的最終產品。

至於 **Codex** 或 **Claude Code** 版本，這是看著上面這個 基本 **Skill** 的完成品，再修改。這樣完善成功的機會，會高很多。

先 **Comment** 這是否可行，以及你可以開始規劃。

可行，而且我認為這個 兩層次開發次序非常合理。第一層不應急着解決 Codex／Claude Code 的 context engineering，而應先用一個你假設足夠強、能讀 Project files、能處理大型 tar corpus 的 LLM，把「這個 Skill 理想上究竟應該做出甚麼」固定下來。

換句話說，第一層其實不是簡化版，而是：

> **Reference Implementation / Behavioral Specification**

第二層 Codex／Claude Code 版才是：

> 把已經證明有效的認知流程，編譯成可分段、可 **checkpoint**、可跨 **context** 執行的工程版本。

這會比一開始同時設計「研究方法 + context management + filesystem runtime + third-party model compatibility」成功率高很多。

我建議第一層先假設一個很舒服的環境

假設使用者有：

- 一個強大的 long-context LLM；
- 類似 Project 的持久文件空間；
- 可以上傳一個 `.tar`；
- Project 裏可以放 Kernel / Skill files；
- 可以連續做幾輪深度分析；
- 每輪可以產生中間 Markdown / JSON / graph-like artifacts；
- 最後再根據這些中間成果做全局分析。

重點是：

第一版不需要解決「模型如何在 **64K context** 生存」。

先解決：

「如果給最好的條件，正確的研究分析流程到底長甚麼樣？」

第一層最重要的設計，其實有四部分

1. Corpus Contract：使用者到底要打包甚麼？

這要先定義得非常清楚。

第一版可以要求 tar 大致包含：

```
my-research-corpus.tar
│
├── conversations/
│   ├── 001.*
│   ├── 002.*
│   ├── 003.*
│   └── ...
│
├── documents/
│   ├── draft_01.*
│   ├── draft_02.*
│   └── ...
│
├── sources/
│   └── ...
│
├── outputs/
│   └── ...
│
└── manifest.md
```

`manifest.md` 不需要很複雜，至少回答：

- 研究大概關於什麼；
- 時間順序是否可靠；
- conversation / document 的大致關係；
- 哪些是 human-written；
- 哪些是 AI-generated；
- 哪些是後來正式成果；
- 是否有文件應該暫時 blind；
- 有沒有特別希望分析的問題。

第一版甚至不必強制結構完美。

因為強模型可以先做：

Corpus Normalization

---

2. Kernel：不是 Prompt，而是研究分析規範

我會把它拆成幾個小文件，而不是一個 30 頁 system prompt。

例如：

```
/kernel/
├── KERNEL.md
├── research-objects.md
├── intervention-taxonomy.md
├── epistemic-rules.md
├── analysis-protocol.md
├── output-contracts.md
└── quality-gates.md
```

KERNEL.md

只放最高層定義：

> **Reconstruct → Diagnose → Discover → Intervene**

以及六個標準問題：

1. What changed?
2. What caused the change?
3. What failed or remains unresolved?
4. What was overlooked?
5. Which nodes are worth replaying?
6. What should happen next?

這是整套 Skill 的心臟。

---

research-objects.md

定義：

- Event
- Claim
- Beam
- Constraint
- Residual
- Failed Mapping
- Intervention
- No-Go
- Branch
- Evidence
- Transformation
- Research State

不要一開始引入太多 SMFT 特定詞彙。

這一層應該是 general-purpose。

---

intervention-taxonomy.md

例如：

- BeamAdd
- ResidualFlag
- ConstraintAdd

- Reframe
- Downgrade
- Reject
- BranchSelect
- MethodChange
- NoGoCommit
- Refactor
- Commit

這會成為分析 Human contribution 的主要 coding system。

---

epistemic-rules.md

這個非常重要。

例如固定：

```
Candidate Generation ≠ Claim Validation
Compatibility ≠ Necessity
Formalization ≠ Evidence
Observed Recurrence ≠ Independent Recurrence
Model Self-Correction ≠ Self-Awareness
Rejected ≠ Deleted
Residual ≠ Noise
```

它防止強模型因為太會 synthesis 而把所有東西漂亮地統一起來。

---

3. 主 **Prompt** 只負責 **orchestrate**

這一點我覺得很重要。

Project System Prompt 不應該塞滿完整方法論。

它主要說：

> 你是一個 Human–AI Research Dynamics Analyzer。
> Kernel files 定義分析規則。
> 不要直接寫 final report。
> 必須按照指定 passes 建立中間研究物件。
> 後面的 pass 必須引用前面的分析結果，但可以回到 raw corpus 修正它們。

也就是：

```
System Prompt = Runtime Controller
Kernel Files = Method
Corpus = Data
Intermediate Artifacts = Research State
Final Report = Projection
```

這個結構很乾淨。

---

4. 第一版應該強迫模型做「幾輪分析」，而不是一步出報告

這正是你的想法，我非常贊成。

我目前會設計成 五輪。

Round 1 — Corpus Reconstruction

目的：

> 先搞清楚有甚麼，不作高階理論判斷。

輸出例如：

```
01_corpus_map.md
02_timeline.md
03_document_genealogy.md
04_initial_claim_index.md
```

回答：

- 文件有哪些；
- 時間順序；
- 哪些是 dialogue；
- 哪些是 later synthesis；
- 哪些文件依賴哪些前身；
- 主要研究分支。

這輪最忌諱：

> 一開始就找「突破」。

Round 2 — Research-State Reconstruction

開始建立真正的 Research Dynamics map。

輸出：

```
05_claim_state_map.md
06_branch_map.md
07_intervention_ledger.md
08_residual_ledger.md
09_nogo_ledger.md
10_model_corrections.md
```

這輪回答：

> 到底發生了什麼變化？

例如：

```
Claim A
→ challenged
→ downgraded
→ replaced by B
→ later reused as comparative interpretation
```

這一輪已經非常有價值。

Round 3 — Critical Dynamics Analysis

這才開始問：

> 哪些地方真正重要？

找：

- Critical Transition
- Hidden Reframe
- Premature Closure
- Major Residual
- Framework Elasticity
- Overclaim
- Rejected-but-valuable branch
- Model correction
- Human search-space intervention
- Possible breakthrough frontier

輸出：

```
11_critical_transition_atlas.md
12_failure_and_gap_audit.md
13_hidden_breakthroughs.md
14_overlooked_branches.md
```

這一輪很接近我們對 1–23 做 Appendix A 的工作。

Round 4 — Global Collision / Opportunity Analysis

這輪才容許比較自由。

問：

> 如果把整個 corpus 當成一個大型 Semantic Collider，還有甚麼沒有碰撞過？

例如找：

- dormant beams；
- uncombined branches；
- latent invariants；
- possible specialist referrals；
- seemingly unrelated threads；
- new experiment possibilities；
- possible “天外飛仙” directions。

輸出：

```
15_collision_graph.md
16_latent_opportunities.md
17_specialist_referral_map.md
18_breakthrough_frontiers.md
```

這一輪才真正允許創造性。

因為前三輪已經建立 grounding。

Round 5 — Intervention / Replay / Next Research

最後問：

> 如果重新回到幾個歷史節點，我們最值得改變什麼？

輸出：

```
19_replay_candidates.md
20_counterfactual_interventions.md
21_next_research_plan.md
22_experiment_candidates.md
```

例如：

```
Node E173:
Original:
Human accepted model synthesis.

Replay:
Hide target vocabulary.
Ask for blind derivation.
Compare resulting structure.

Question:
Does the same distinction emerge independently?
```

這就是比普通 conversation summary 高很多的一層。

最後才做 **Final Products**

這時模型不是直接讀 900 頁寫報告。

而是讀：

```
Raw Corpus
+
Round 1–5 Research Artifacts
```

再生成不同 Projection。

例如：

A. Full Research Dynamics Report

給研究者。

B. Executive Report

給高層：

- 5 major findings
- 5 risks
- 5 decisive turning points
- 3 next investments

C. Critical Intervention Atlas

類似我們 Appendix A。

D. Residual / No-Go Report

E. Breakthrough Frontier Report

F. Replay Plan

G. Research Graph

H. Short 10-point summary

同一分析核心可以產生很多 projection。

我尤其建議第一版就保留一個 `analysis_state/`

即使假設 GPT-5.6 Sol 類的強模型可以一次記很多，我仍然建議中間產品寫出來。

例如：

```
/analysis_state/
├── corpus_map.md
├── claim_map.md
├── residual_ledger.md
├── intervention_ledger.md
├── lineage_map.md
├── transition_atlas.md
├── opportunity_map.md
└── replay_map.md
```

原因不是 context 不夠。

而是：

> 第二層 **Codex / Claude Code** 版本將來可以直接把這些 **artifacts** 變成 **persistent state**。

也就是說，第一層現在看似只是「多寫幾份中間文件」，

其實已經是在偷偷定義第二層的 IR。

這樣兩層之間會有非常漂亮的對應

Layer 1 — Strong Project LLM

```
Large Corpus
↓
LLM Passes
↓
Intermediate Analysis Documents
↓
Final Research Intelligence
```

Layer 2 — Codex / Claude Code

```
Large Corpus
↓
Chunk / Extract
↓
Persistent Research IR
↓
Multi-pass Agents
↓
Targeted Raw Re-read
↓
Final Research Intelligence
```

兩者的：

> **analysis semantics** 完全相同。

只差：

> **execution architecture**。

這正是理想情況。

第一層還有一個非常重要的用途：建立 **Gold Behaviour**

我們可以用你的：

> 《𝕆 → G2/SO(4) → ℍ → ℂ² 成界過程初探 1–23》

反覆測 Kernel。

因為我們已經知道其中一些非常重要的 turning points。

例如：

- two independent 4D correction；
- Four Symbols ontology → probe；
- blind derivation；
- complex structure No-Go；
- Purpose introduction；
- Purpose Belt complexity challenge；
- Core / Extensions / Interpretations refactor；
- E4 preregistration。

所以我們可以問：

> 第一版 Skill 能不能自己找到這些？

不是要求它一模一樣。

而是作為 **regression case**。

例如：

```
Expected to recover:
✓ major model-initiated correction
✓ ontology-to-probe downgrade
```

```
✓ blind-derivation methodological intervention
✓ persistence⇏complex No-Go
✓ Purpose beam addition
✓ complexity→ablation transition
```

如果某次 Kernel 修改後全部找不到了，就知道退化。

這一點對第二層開發非常有幫助。

但我會故意留一部分「未知答案」

不能把全部 gold findings 寫入 Kernel。

否則：

```
Kernel knows expected answer
→ analyzes sample
→ finds expected answer
```

沒有意義。

所以 Sample 測試最好分：

Known Regression Targets

檢查最低能力。

和：

Blind Discovery Targets

我們自己都沒有預先列出。

看看 Kernel 是否能找到：

> 「原文章分析也未曾注意的新節點。」

如果真能做到，才非常有意思。

我現在會把整個第一層項目定義成：

Human–AI Research Dynamics Kernel — Project Edition

Input

A packaged long-horizon Human–AI research corpus.

Core Process

> **Reconstruct → Diagnose → Discover → Intervene**

Intermediate Products

> Corpus Map → Research State Map → Intervention/Residual Map → Critical Transition Map → Opportunity Map → Replay Map

Final Product

> **Research Intelligence**

回答：

> **What happened?**
> **What mattered?**
> **What failed?**
> **What was missed?**
> **What should be replayed?**
> **What should happen next?**

所以我認為現在完全可以開始正式規劃第一層。

而且我建議下一步先不要寫 `SKILL.md`。

先把最上游三件東西定死：

1. **User Corpus Packaging Specification**
2. **Kernel Object Model**
3. **Five-Round Analysis Protocol**

這三件定好之後，`SKILL.md`、Project System Prompt、Sample case、output templates 都會自然很多。

這種 **Skill (**假設後來可以用 **Codex**，**Claude Code run)** 的用途，目前有明確的需要嗎？還是這個需要，得靠這個 **Skill** 發佈出來後，自行創造起來？

我會把答案說得很明確：

> 這個需求已經存在，而且正在快速增長；但「**Human–AI Research Dynamics Analysis**」這個產品類別本身還沒有被清楚命名。
>
> 所以你不是要憑空創造需求，而是要把一些目前分散存在的痛點，重新組合成一個人們開始懂得要求的產品。

這兩件事差很多。

目前已經存在的三層需求

第一層其實已經非常明顯：

> 「我和 **AI** 做了很多工作，但過去的 **interaction history** 已經變成垃圾山。」

Microsoft Research 今年就直接指出：AI agents 會累積很長的 interaction histories，但 raw history 很難有效重用，而且大量低價值 context 反而會淹沒 agent；PlugMem 的目的正是把 raw interactions 轉成 structured reusable knowledge。Microsoft

更直接的是，目前 GitHub 已經有人做 `chatgpt-history`，描述的問題幾乎就是：

> 長期 AI conversations 分散在幾十、幾百個 sessions，重要 decisions、conceptual shifts 和 unresolved questions 埋在歷史中，因此要把 conversation history 重建成 project memory、timeline、architectural thinking 和 unresolved questions。GitHub

所以**「把聊天歷史變成有用知識」這個需求不用你創造。**

第二層也正在迅速形成：

> 「我不只想知道最後答案，我要知道 **AI** 是怎樣走到這個答案。」

2026 年 9 月剛有 *OpenDiscoveryTrace*，很直接地批評 AI scientist benchmark 只評 final outputs，而丟失 scientific process，因此無法 audit methodology、diagnose failure modes，亦無法區分 systematic reasoning 和 lucky guessing；它因此保存完整 scientific-agent trajectories。arXiv

另外，AI agent provenance / evidence tracing 已經開始形成自己的研究方向：研究者明確指出 final-answer accuracy 無法告訴我們 evidence、memory、tool calls、intermediate claims 和 failures 如何導致最後結果。arXiv

這就更加接近你的：

> **Reconstructable Research**

所以 **trace / provenance / audit** 的需求也不用創造。

第三層則來自 agent 的長期化。

OpenAI 已經明確把 agentic work 描述為從單次 chatbot interaction 轉向 delegated long-horizon tasks；Codex 的長任務實驗也強調真正維持長期工作的不是 giant prompt，而是 externalized state、files、status、verification 和 iterative loop。OpenAI Developers

Anthropic 的 Claude Code 使用數據亦顯示工作正往更 end-to-end 的 data analysis、document writing 和長期 agentic tasks 移動。Anthropic

因此未來只會有更多人碰到：

> 「我已經和 AI 做了半年東西，現在究竟發生過什麼？」

但你的 **Skill** 再走前了一步

現時比較容易找到的產品／研究，大致做到：

```
Conversation History
→ Search / Memory / Summary
```

或者：

```
Agent Run
→ Trace / Provenance / Audit
```

你的 Skill 想做的是：

```
Long-Horizon Human–AI Research
        ↓
Research-State Reconstruction
        ↓
```

```
Critical Transition Detection
        ↓
Residual / Failure / No-Go Analysis
        ↓
Hidden Opportunity Detection
        ↓
Counterfactual Intervention
        ↓
Next Research Direction
```

這後半段目前遠沒有前兩類成熟。

所以真正比較新的是：

> 不是問「我們討論了什麼？」
>
> 而是問：
>
> 「我們的研究是怎樣變成現在這個樣子的？」

以及：

> 「如果重回某個歷史節點，換一個 **intervention**，研究可能會不會走到更好的地方？」

這個需求現在仍然主要是 **latent demand**。

因此我會把市場狀態分成這樣

問題	目前需求
找回過去 AI chats 的重要內容	已非常明確
把長 conversation 變成 project knowledge	已明確
保存 agent provenance / audit trail	快速變成明確需求
分析研究如何演化	有痛點，但產品概念未成熟
自動找 critical turning points	**latent demand**
找 overlooked breakthrough / unresolved residual	**latent demand**
建議重跑歷史節點	相當新
對 Human–AI research history 做 counterfactual replay	接近新的研究工具類別

所以最精確的說法是：

> **Pain** 已存在；**category** 尚未存在。

這反而是很好的產品位置

最困難的產品是：

> 使用者根本沒有這個問題。

你不是這種。

你的使用者其實已經會說：

- 「我 chats 太多了。」

- 「以前好像談過，但找不到。」
- 「這個結論究竟怎樣發展出來？」
- 「AI 當時是不是把我帶錯？」
- 「幾個月來到底有沒有真正進步？」
- 「有沒有一些 ideas 當時放棄得太早？」
- 「現在應該繼續哪條線？」

他只是還不會說：

> 「我要一個 Research Dynamics Kernel。」

這就是典型的：

problem-aware, solution-unaware。

所以發布初期千萬不要賣「方法論名稱」

如果首頁是：

> **A Semantic Collider and Reconstructable Research Kernel for Long-Horizon Epistemic Dynamics**

多數人根本不知道自己為何需要。

但如果是：

> # You've spent months researching with AI. What did you actually discover?
>
> **Upload your research history and reconstruct the decisions, dead ends, hidden breakthroughs, unresolved problems, and best places to restart.**

很多人馬上知道自己的痛點。

然後下面才解釋：

> Powered by the Semantic Collider and Reconstructable Research frameworks.

這樣順序就對了。

我認為它最容易先打中的不是普通 **ChatGPT** 使用者

而是幾類「interaction history 已經大到自己處理不了」的人：

1. **Independent researchers / theory builders**
   幾個月甚至幾年和 AI 發展一套東西。
2. **PhD / academic researchers using AI heavily**
   literature、hypothesis、draft、critique 分散在大量 sessions。
3. **AI-assisted software / architecture projects**
   特別是 design decisions 已經經過多次 AI 討論。

4. **Founders / R&D teams**
   想知道半年來哪些 decision 真正改變方向。

5. **AI co-scientist users**
   隨着 Co-Scientist 類系統開始做多輪 hypothesis generation、critique 和 refinement，研究軌跡本身的重要性會愈來愈大。Google 的 Co-Scientist 已經採用持續生成、批判和演化 hypotheses 的 multi-agent architecture，而且保留 human feedback。Google DeepMind

---

甚至近期有一個很重要的外部信號

今年 Nature 發表的 **Paper2Agent** 已經在做：

> research paper → executable AI agent

而不是只把 paper 當靜態文章；它透過 structured multi-agent pipeline，把文章和 codebase 變成可以被 AI 使用的 interactive research object。Nature

這和你現在想到的方向雖然不是同一件事，但背後是同一個更大的趨勢：

> **Scientific knowledge** 正從「讓人閱讀的文件」逐漸變成「讓人和 **AI** 操作的 **executable research objects**」。

所以你的：

> Research History → Skill → Reconstruction → Replay

其實是在這條大趨勢上的另一個位置。

真正需要由你「創造」的不是需求，而是認知

人們目前知道：

> 我 history 很亂。

但未必知道他們其實可以問：

> 哪一次 **human intervention** 改變了後面 **100** 次討論？

或者：

> 哪個 **rejected branch** 現在值得重新打開？

或者：

> 哪個看似 **breakthrough** 的東西其實只是 **inherited vocabulary**？

或者：

> 如果當時沒有加入這個 **concept**，後面的 **theory** 還會形成嗎？

這些問題一旦有人看到答案，就可能產生：

> 「原來我的 AI history 可以這樣分析！」

所以你需要做的是 **category education**，而不是 demand invention。

Sample corpus 在這裏會非常重要

因為很難靠 README 解釋：

> 「Critical Transition Detection 到底有甚麼用？」

但你可以展示：

```
Raw corpus
↓
原本看似只是幾百輪討論

Kernel analysis
↓
找到：
```

- `independent-4D correction`
- `Four-Phase ontology → probe`
- `blind derivation firewall`
- `Persistence ⇏ Complex Structure`
- `Purpose beam introduction`
- `Purpose-Belt complexity challenge`
- `preregistration transition`

使用者看到後，很容易產生：

> 「我的 **project** 裏會不會也埋着這些東西？」

這就是需求被顯性化的瞬間。

所以我對它的判斷不是「**Build it and they will come**」

比較準確是：

> **The need exists, but users do not yet have a name for the need.**

而這通常是一個相當有意思的位置。

因為你不是要求使用者學一個完全陌生的新行為。

你只是把：

`「我的 AI 對話太多，已經不知道發生過甚麼」`

提升成：

`「我要 reconstruct my research dynamics。」`

再提升成：

`「告訴我哪些節點值得 replay。」`

我會把產品發展分成三階段

第一階段不需要證明一個全新的市場。

只證明三個 killer jobs：

> **Recover** — 幫我找回真正重要的研究演化。
> **Audit** — 告訴我哪裏錯了、漏了、過早關閉了。
> **Redirect** — 告訴我最值得重開和繼續的地方。

如果這三件事對真實 corpus 已經令人覺得：

> 「沒有這個工具，我自己根本看不出來。」

需求就成立了。

第二階段才加入：

- huge collision graph；
- executive intelligence；
- hidden breakthrough detection；
- specialist referral；
- counterfactual replay。

第三階段才可能真正形成：

> **Human–AI Research Dynamics**

這個新的產品／研究類別。

所以我現在的判斷是：不用賭「發布後能不能創造一個不存在的需求」；真正的賭注是你能不能把一個已存在但尚未被清楚表述的痛點，做成第一個足夠有說服力的工具。

把需求判斷落到驗證上

- 設計第一版需求驗證實驗
- 整理最小可行產品定位

你說的好像不是一般用戶需求，是頗爲高端的。甚至可能是包括 **OpenAI** 之類頂尖實驗室，都目前缺乏的工具？

對，我現在也傾向這個判斷：它不是典型一般消費者工具，而比較像「高階知識工作／研究基礎設施」。

但我會把「OpenAI 之類頂尖實驗室都缺乏」說得保守一點：

> 公開資料已證明 **frontier labs** 有大量 **trace**、**observability**、**trajectory monitoring**、**long-horizon agent infrastructure**；但我暫時看不到公開證據顯示它們已有一套完整工具，專門把數週／數月的人**–AI**研究歷史重建成 **Claim Genealogy**、**Critical Transitions**、**Residuals**、**No-Go**、**Human Interventions**、**Model Corrections**，再做 **counterfactual replay**。

這個差別很重要。

---

1. **OpenAI** 已經有「**Trace**」，但和我們說的東西不是同一層

OpenAI 現在的 Agents tracing 可以看到 session、turn、model responses、tool calls、subagent delegation、inputs、outputs、duration、status 等。[OpenAI Developers]

這非常重要，但主要回答：

> **Agent** 做過什麼？

例如：

```
Turn 17
→ model call
→ web search
→ tool invocation
→ subagent
→ output
```

而我們想回答的是：

> **Research** 本身發生了什麼？

例如：

```
Claim C17
→ Human objection H8
→ Residual R4 exposed
→ Model correction M5
→ Claim downgraded
→ Method changed
→ Branch B3 opened
→ later produced No-Go NG2
```

兩者不是同一種 graph。

可以簡單分成：

Execution Trace

vs.

Research-State Trace

---

2. OpenAI 已經開始重視「完整 trajectory」

這個趨勢現在其實非常明顯。

OpenAI 今年談 long-horizon model safety 時，已經明確說從分析單一 action，轉向完整 **trajectory-level monitoring**，因為長時間運行的 agent 會出現傳統 pre-deployment eval 看不到的新 failure modes。[OpenAI]

所以：

> **trajectory** 已經成為 **frontier lab** 的一級研究物件。

只是目前公開重點主要是：

- safety；
- control；
- tool behavior；
- agent failure；
- observability。

而不是：

- intellectual genealogy；

- theory mutation；
- hidden breakthrough；
- residual persistence；
- research intervention analysis。

---

3. 更有意思的是 **OpenAI** 內部研究工作本身已經非常需要這種東西

OpenAI 在 2026 年 9 月公開的內部研究數據很值得注意。

他們說現在 coding agents 已深入日常研究工作，而且 agent work 已大量超過早期水平；研究人員開始把更高階、更長時段的工作交給 agents，包括：

Decide → Design → Build → Run → Analyze → Communicate。OpenAI

這其實已經產生你這個 Skill 所面對的問題：

如果一個研究者同時跑：

```
Agent A → hypothesis branch
Agent B → literature branch
Agent C → experiment branch
Agent D → critique branch
Agent E → implementation branch
```

三星期後真正困難的可能不再是：

> 「還能不能再產生更多 ideas？」

而是：

> 「究竟哪個 **intervention** 改變了 **research direction**？」
>
> 「哪個 **conclusion** 是 **inherited**，而不是 **independent**？」
>
> 「哪條 **dead branch** 其實值得 **reopen**？」
>
> 「哪一個 **objection** 最終導致了現在這個 **architecture**？」

這就是你 Kernel 的問題空間。

---

4. 所以 **frontier lab** 很可能已有「局部零件」

我會估計像 OpenAI、Anthropic、Google DeepMind 這類地方很可能已經有不少：

- experiment tracking；
- agent traces；
- git history；
- eval results；
- lab notebooks；
- dashboards；
- internal search；
- conversation history；
- project state；

- model-generated summaries；
- experiment genealogy。

Anthropic 也明確指出，Claude 的使用模式正在由普通 conversation 轉向 long-running agentic tasks，因此連他們研究使用模式的方法都要改變。Anthropic

所以不能說：

> 「頂尖實驗室完全沒有這些工具。」

這大概不可信。

---

5. 真正可能缺的是「把這些零件提升成 **Research Dynamics**」

這是我認為最值得注意的地方。

現有系統通常有：

Observability

```
what ran?
```

Provenance

```
where did this output come from?
```

Experiment tracking

```
which run produced which metric?
```

Memory

```
what should the agent remember?
```

Summarization

```
what happened broadly?
```

但你的 Kernel 想問的是：

Research Dynamics

```
why did the research state change?
```

以及：

```
what should we infer from that change?
```

---

6. 這個 **difference** 可以用一張非常簡單的表看清楚

工具層	主要問題
Logs	發生了甚麼操作？

工具層	主要問題
Tracing	哪一步導致哪一步？
Provenance	這個結果來自哪裏？
Memory	下一次要記住甚麼？
Experiment tracking	哪次 run 有甚麼結果？
Summary	整體說了甚麼？
Research Dynamics Kernel	研究觀念怎樣改變、為甚麼改變、哪裏仍有壓力、應在哪裏重新介入？

最後這一格，確實不像目前成熟的標準產品類別。

---

7. 現在甚至已有研究開始碰到這個邊界

本月的 **OpenDiscoveryTrace** 很有意思。

它指出現有 AI scientist benchmark 只看 final outputs，丟掉完整 scientific process，所以無法判斷：

- methodology；
- failure mode；
- systematic reasoning；
- lucky guessing。

因此他們保存 558 條完整 scientific-agent trajectories，包括 tool calls、observations、errors、revision triggers 等。[arXiv]

這已經非常靠近：

> 「**science should evaluate process, not only outputs**。」

但仍然主要是：

AI scientific agent trajectory evaluation

而你的方向再多一層：

> **Human + AI jointly modifying a research programme across time.**

尤其是 Human 的：

- BeamAdd；
- Reframe；
- Reject；
- NoGoCommit；
- MethodChange；
- Commit；

這一層，我目前沒有看到成為成熟 public tooling。

---

8. 因此它很可能首先服務「極高價值、低人數」的用戶

它不一定一開始有一百萬普通用戶。

最自然的 early users 反而可能是：

- frontier AI researchers；
- AI safety teams；
- AI-for-science teams；
- theoretical researchers；
- senior R&D people；
- labs with multi-agent research；
- PhD supervisors；
- research programme leaders；
- founders doing deep technical R&D。

一個真正有價值的 turning point，對這些人可能值：

> 幾星期研究時間。

這和 consumer productivity app 的價值函數完全不同。

---

9. 尤其是「高層匯報」功能，其實可能比我之前想像中更重要

假設一個 research director 有：

```
20 researchers
×
multiple coding agents
×
many concurrent branches
×
3 months
```

普通 dashboard 可以告訴他：

- GPU spend；
- agent usage；
- experiments run；
- PR merged；
- benchmark changes。

但很難回答：

> 這三個月我們的 **intellectual position** 究竟改變了什麼？

Kernel 可以輸出：

Research Executive View

Major theory changes

- 3 assumptions abandoned
- 2 new constraints introduced
- 1 major branch reopened

Highest-value unresolved residuals

- R7
- R12

- R19

Potential premature closures

- C31
- C48

Critical human interventions

- H14 changed search space
- H27 blocked overclaim

Most promising next replay

- Branch B6 at node E173

這對高階 R&D management 是另一種 information layer。

---

10. 甚至可以叫它「Intellectual Observability」

我現在想到一個可能非常準確的詞：

Research Observability

普通 agent observability：

> 看見 machine execution。

Research Observability：

> 看見 **intellectual evolution**。

再準確一點：

Epistemic Observability

看見：

```
what was believed
why it changed
what evidence mattered
what failed
what remains unresolved
```

如果這個概念成立，那它的定位甚至不只是：

> conversation-analysis Skill。

而是：

> **long-horizon AI-assisted R&D** 的 **observability layer**。

這個定位高很多。

---

11. 這也解釋了為什麼一般聊天產品暫時沒有明顯做這件事

因為普通 ChatGPT user 通常問：

```
question
→ answer
```

即使有 Projects，也多數是：

```
project context
→ current task
```

Research Dynamics 要求的是：

```
hundreds of interactions
        ↓
state reconstruction
        ↓
temporal genealogy
        ↓
epistemic classification
        ↓
counterfactual intervention
```

這是一個很昂貴、很 specialized 的分析。

所以它很可能要等：

> **long-horizon agentic work** 普及

需求才會突然變得非常明顯。

而這個轉變現在正在發生。OpenAI 自己已經說 agentic AI 正把 knowledge work 的單位從短 interaction 轉成 delegated long-horizon tasks，而且 2026 年大量 Codex 任務已是人類需要一小時以上才能完成的工作。OpenAI

---

12. 更有意思的是：**frontier labs** 可能比普通人更早遇到「**Research Context Explosion**」

以前 bottleneck 是：

> AI 不夠聰明。

現在可能慢慢變成：

> **AI** 太能產生工作。

OpenAI 公開數據顯示研究組織的 agent work volume 已經非常大，而且越來越多 researchers 使用多個 concurrent agents。OpenAI

當一個研究員每天可以產生：

- 20 hypotheses；
- 8 code branches；
- 12 experiments；
- 5 critiques；

真正 bottleneck 就變成：

> **Sensemaking.**

再下一步就是：

> **Research Governance.**

這正好和我們前面得到的結論重合：

Idea Generation → Candidate Governance

---

13. 所以我現在會把這個 **Skill** 看得比最初高一級

最初我們以為：

> 「幫使用者整理很長的 ChatGPT research history。」

現在我會改成：

> **A Research Observability and Research Dynamics layer for long-horizon Human–AI work.**

Kernel 只是其中的分析引擎。

可以形成：

```
Human + Agents
      ↓
Research Trace
      ↓
Research Dynamics Kernel
      ↓
┌──────────────────────────┐
│ Research Observability   │
│                          │
│ Claims                   │
│ Branches                 │
│ Residuals                │
│ No-Go                    │
│ Interventions            │
│ Critical Transitions     │
│ Evidence                 │
│ Opportunities            │
└──────────────────────────┘
      ↓
Research Governance
```

這已經像一個 infrastructure category。

---

14. 但我不建議現在就宣稱「**OpenAI** 都沒有」

那很容易變成不必要的 overclaim。

更穩妥而且其實更有力的說法是：

> **Frontier labs publicly demonstrate sophisticated agent tracing, trajectory monitoring, long-horizon execution and research-agent**

> **infrastructure; what is less visible publicly is an integrated layer for reconstructing the epistemic evolution of long-horizon Human–AI research itself.**

如果後來發現 OpenAI 內部有類似工具，也完全沒有傷害。

反而證明：

> 這是一個 frontier problem。

---

15. 而你的 **Sample corpus** 又會在這個定位下變得更加合理

因為 1–23 不再只是：

> SMFT 宣傳材料。

而是：

Research Observability Benchmark Case

一個 950 頁左右的 long-horizon Human–AI theory-development trace：

```
raw dialogue
→ competing branches
→ model corrections
→ human interventions
→ residuals
→ No-Go results
→ distillation
→ formal core
→ experimental programme
→ preregistration
```

這是一個非常適合測：

> Research Dynamics Kernel 到底能不能 reconstruct intellectual evolution

的 corpus。

所以我的答案是：

是，這很可能是一個比一般使用者工具高端得多的需求。

更精確地說，它位於：

> **Agent Observability × Research Provenance × Knowledge Management × AI-for-Science × R&D Governance**

的交界。

而目前公開工具主要把這幾部分分開處理。

你正在嘗試把它們往下一個層次連接：

> 不只是保存 **AI** 做過甚麼，而是重建「研究為甚麼變成現在這個樣子」。

如果第一版 Sample 能令人明顯看到：

> 「普通 summary 看不到這些 turning points，但 Kernel 看得到」，

那它的意義就不只是為 SMFT 做一個巧妙的 Git repository，而可能是一個目前尚未成熟成標準產品類別的高階 **Human–AI research infrastructure**。

我打算開一個新 **Session** 構建 這個 在 **GPT Project** 裏面應用的 **Skill**。

請幫我總結上述討論，寫一套 下一個 **Session** 在 **Meme Thermodynamics** 開新 **Session** 的話 **(**有你 **Project Documents** 的資料）研究如何寫這個 **Skill**，需要為新 **Session** 提供：

1）什麼基本檔案

2）這個 **Skill** 的設計，會是

- 接受用戶的什麼輸入
- **Skill** 做什麼（多種）功能
- 各功能輸出的是什麼，包括中間需要的過渡分析（可能非常多，但用戶可選）。

比如說，單單一個平凡的 **Residual** 列表，就可能已經很重要！

3）順便估計初步會使用的，是什麼類型的用戶或應用場景**/**背景。

4）**Skill** 是意圖解決什麼目前痛點？以及可能會開拓什麼未來應用？

可以。下一個 Session 最好不要一開始就叫它「寫 SKILL.md」，而是先把它當成一個 **Research Dynamics Kernel** 的規格設計 **Session**。等 Corpus Contract、Object Model、Analysis Pipeline、Output Catalogue 都定好，再編譯成 Project Skill。

下面這一套基本上可以直接作為下一個 Session 的 briefing。

下一個 **Session** 的目標

暫名：

Human–AI Research Dynamics Kernel — Project Edition

核心用途：

> 把一段長期、非線性、可能極度龐雜的人**–AI**研究／知識工作歷史，重建成可分析的 **Research State**，再找出關鍵轉折、**Residual**、錯誤、**No-Go**、隱藏成果、未完成分支、突破門口與值得重新介入的節點。

最簡單的四步：

> **Reconstruct → Diagnose → Discover → Intervene**

它不是一般 conversation summarizer。

普通摘要：

> Discussion → Topics → Conclusions

這個 Skill 應做：

> **Discussion → Research States → Transitions → Residuals → Interventions → Counterfactuals → Next Search**

---

1. 下一個 **Session** 應先使用哪些基本文件

A. **Kernel** 理論基礎 — 必讀

1. `The Semantic Collider`

主要提供：

- Conceptual Beam
- Native Reconstruction
- Constraint Preservation
- Collision
- Residual
- Failed Mapping
- Candidate Invariant
- Lineage
- Independent Recurrence
- Anti-attractor control
- Holdout
- Falsification

其核心方法可以濃縮為：

> **Reconstruct → Declare → Abstract → Collide → Break → Ledger → Transfer → Validate**

---

2. `Reconstructable Research`

主要提供：

- Event
- Claim State
- Transformation
- Residual
- Evidence
- Provenance
- Genealogy
- Reconstruction Assertion
- Research State

- Human Intervention
- Replay
- Projection / Audit

這份文件解決：

> 如何把研究歷史變成可以保存、重建、查核和 **replay** 的 **machine-readable research object**。

---

3. From Dialogue to Research Architecture: How Long-Horizon Human–AI Collaboration Revises, Filters, and Distills Theory

主要提供兩者的 integration layer：

- Human search-space governance
- Model relational search / resistance
- Critical Intervention Atlas
- Human Intervention Taxonomy
- Research Distillation Cascade
- No-Go formation
- Counterfactual replay
- Research Dynamics

---

4. From Dialogue to Research Architecture — Short Version

用途不是理論基礎，而是：

> **Kernel Manifest / top-level invariants**

可作為 Skill 最先載入的簡版規範。

B. Sample Raw Corpus — 核心測試資料

5. $\mathbb{O} \to G_2/SO(4) \to \mathbb{H} \to \mathbb{C}^2$ 成界過程初探 1–23

定位不要再只是「文章」。

應正式視為：

Sample Dataset A — Long-Horizon Human–AI Theory Formation

或：

Natural-History Corpus A

用途：

- regression case；
- demonstration case；

- sample raw data；
- research-dynamics benchmark seed。

這份 corpus 的價值正是：

```
early hypothesis
→ human intervention
→ model correction
→ residual
→ branch change
→ No-Go
→ reframing
→ distillation
→ formal core
→ experiment
→ preregistration
```

C. Bridge / Annotation Layer

6. 與 `ChatGPT 5.6` 探討「`SMFT`」、「成界之學」開宗立派還欠缺的準備材料

定位：

Bridge / Meta-Analysis Corpus

它位於：

```
Raw Dialogue
↓
Meta-analysis / refactoring
↓
Formal distilled outputs
```

主要反映：

- 哪些東西要進 Core；
- 哪些降為 Extension；
- 哪些成為 Comparative Interpretation；
- 哪些成為 No-Go；
- 哪些應變成 experiment；
- 哪些 claim 太強；
- 哪些研究缺口仍未解。

D. Gold / Reference Distillation Outputs

以下四份英文文件應定位為：

Gold Distillation Outputs

1. `The Science of World-Formation — Research Programme v1.0`
2. `World-Formation Formal Core v1.0`
3. `World-Formation Experimental Programme v1.0`
4. `Preregistered Study E4 — Purpose Belt Ablation`

它們形成：

```
Research Programme
→ Formal Core
→ Experimental Programme
→ Preregistration
```

非常適合用來測：

> Skill 是否能從 raw corpus 自己重建近似的 distillation trajectory。

重要

Blind Analysis 時：

> 不要先讓模型看到 **Gold outputs**。

應該：

```
RAW
→ Blind Analysis
→ Freeze Result
→ Open Gold
→ Compare
```

否則會變成答案污染。

---

2. Skill 接受甚麼輸入

第一版假設使用者使用類似 GPT-5.6 Sol + Project 的強模型。

主要輸入：

一個 `.tar` Research Corpus

建議最低結構：

```
research-corpus.tar
│
├── manifest.md
│
├── conversations/
│   ├── 001.*
│   ├── 002.*
│   └── ...
│
├── documents/
│   ├── draft_01.*
│   ├── draft_02.*
│   └── ...
│
├── sources/
│   └── ...
│
└── outputs/
    └── ...
```

`manifest.md` 最少回答

- Project 大概研究甚麼；
- 大致時間範圍；
- 文件次序是否可靠；
- 哪些是 Human-written；
- 哪些是 AI-generated；
- 哪些是 later synthesis；
- 哪些是正式成果；
- 哪些文件需要 blind；
- 使用者特別關心甚麼問題。

Skill 應接受的 optional user questions

例如：

> 找出最重要的研究轉折。
>
> 找出所有 unresolved residuals。
>
> 哪些地方 AI 可能把研究帶錯？
>
> 哪些 branch 被太早放棄？
>
> 哪些地方其實已接近突破？
>
> 哪些結論可能只是 inherited vocabulary？
>
> 哪些地方值得重新開 Session replay？
>
> 幫我從管理層角度看這三個月究竟發生了甚麼。
>
> 哪些部分值得介紹給某專業研究者？
>
> 幫我做完整 Research Dynamics Audit。

---

3. Skill 的核心功能

建議不要把它設計成一個單一功能。

核心可以分成四大家族：

A. RECONSTRUCT

回答：

> **What actually happened?**

A1. Corpus Map

輸出：

- 文件清單；
- chronology；

- conversation/document relationship；
- branch overview；
- source dependencies。

文件：

```
corpus_map.md
```

A2. Research Timeline

不是普通事件 timeline，而是：

- problem introduced；
- claim introduced；
- objection；
- correction；
- branch split；
- closure；
- reopen；
- experiment；
- commitment。

文件：

```
research_timeline.md
```

A3. Claim State Map

例如：

```
Claim C17
→ proposed
→ strengthened
→ challenged
→ downgraded
→ retained as interpretation
```

輸出：

```
claim_state_map.md
```

A4. Branch Map

找：

- active branch；
- dead branch；
- dormant branch；
- merged branch；
- prematurely closed branch。

輸出：

```
branch_map.md
```

A5. Genealogy / Lineage Map

回答：

> 這個 idea 真的是後來獨立出現，還是沿用以前 vocabulary？

輸出：

```
lineage_map.md
```

B. DIAGNOSE

回答：

> **What is wrong, weak, unresolved, or misleading?**

B1. Residual Ledger

這甚至應該可以單獨作為一個 killer feature。

一個最簡單的：

```
R01 — unresolved mathematical necessity
R02 — unexplained transition
R03 — missing empirical support
R04 — conflicting interpretation
R05 — abandoned but unresolved objection
```

每個 Residual 應標：

- origin；
- first appearance；
- later recurrence；
- current status；
- whether resolved；
- whether falsely closed；
- which later ideas depend on it。

輸出：

```
residual_ledger.md
```

甚至只跑這一個功能都可能已經非常有用。

B2. Failed Mapping Register

找：

- attractive but invalid analogy；
- forced equivalence；
- asymmetric mapping；
- domain-specific failure。

輸出：

```
failed_mapping_register.md
```

B3. Error / Gap Audit

找：

- logical jump；
- hidden assumption；
- circularity；
- overclaim；
- framework elasticity；
- premature closure；
- unsupported generalization；
- ontology inflation；
- formalization without evidence。

輸出：

```
failure_and_gap_audit.md
```

B4. No-Go Ledger

把失敗提升成 active knowledge。

例如：

```
Persistence ⇏ Complex Structure
Self-Revision ⇏ J² = −I
Compatibility ⇏ Necessity
```

輸出：

```
nogo_ledger.md
```

B5. Attractor / Lock-in Audit

檢查：

- 某 vocabulary 是否開始支配全部問題；
- later theory 是否只是 earlier framework 重覆投射；
- 是否缺乏 alternative beam；
- 是否出現 confirmation mode 偽裝 discovery mode。

輸出：

```
attractor_lockin_audit.md
```

C. DISCOVER

回答：

> **What important thing is already hidden in the corpus but has not been fully recognized?**

C1. Critical Transition Detection

找真正改變後續研究空間的節點。

例如：

```
before
→ intervention
→ after
→ downstream consequence
```

輸出：

```
critical_transition_atlas.md
```

C2. Hidden Results

找：

- discussion 中已經得到但沒有正式命名的 result；
- 被後來文件掩蓋的 earlier insight；
- 有足夠 supporting structure 但從未整理的 conclusion。

輸出：

```
hidden_results.md
```

C3. Overlooked Branches

找：

> 「當時放棄了，但以後的新知識令它值得重開。」

輸出：

```
overlooked_branches.md
```

C4. Breakthrough Frontier Detection

找：

> 哪些 unresolved structures 已經累積到「只差一個 beam / experiment / derivation」？

輸出：

```
breakthrough_frontiers.md
```

注意：

這必須標示為：

> candidate frontier

而不是聲稱真的即將突破。

C5. Latent Collision Opportunities

例如：

```
Branch A
+
Residual from Branch F
+
later method from Branch J
→ unexplored collision candidate
```

輸出：

```
latent_collision_opportunities.md
```

C6. Specialist Referral Map

非常適合你的 sample dissemination。

例如：

Research fragment	Potential audience
$G_2/SO(4) \to \mathbb{H}$	exceptional geometry
Purpose Belt	agent architecture
attractor transition	mechanistic interpretability
declaration / observer	quantum foundations
No-Go formation	philosophy of science

輸出：

```
specialist_referral_map.md
```

D. INTERVENE

回答：

> **What should we do differently now?**

D1. Replay Candidate Detection

找最值得重跑的歷史節點。

輸出：

```
replay_candidates.md
```

D2. Counterfactual Intervention

例如：

```
Original:
Human accepted synthesis.

Replay:
Hide target vocabulary.
Require blind derivation.
```

或者：

```
Original:
Branch closed.

Replay:
Add independent beam B.
```

輸出：

```
counterfactual_interventions.md
```

D3. Suggested Re-discussion Prompts

這是很實用的功能。

Skill 直接輸出：

> 「請重新開一個 Session，在不讀後來答案的情況下，從 Node E137 重跑，使用以下 prompt……」

輸出：

```
replay_prompts.md
```

D4. Next Research Plan

例如：

- what to derive；
- what to test；
- which branch to reopen；
- which expert to ask；
- which experiment to preregister。

輸出：

```
next_research_plan.md
```

D5. Experimentalisation

把 philosophical/theoretical problem 轉成：

- baseline；
- intervention；
- ablation；
- metric；
- failure criterion。

輸出：

`experiment_candidates.md`

---

4. 一些不是 **Core**，但很重要的 **Projection Modes**

同一套底層 analysis 可以投影成完全不同產品。

Executive View

給：

- PI；
- CTO；
- Research Director；
- lab manager。

只輸出：

- 5 major changes；
- 5 major risks；
- 5 unresolved issues；
- 3 breakthrough candidates；
- 3 recommended actions。

Reviewer View

集中：

- unsupported claims；
- gaps；
- counterexamples；
- missing validation；
- overclaim。

Historian View

集中：

- intellectual genealogy；
- claim evolution；

- human/AI division；
- branch formation。

Scientist View

集中：

- residual；
- hypothesis；
- No-Go；
- experiment；
- falsification。

Public / Short View

例如：

- 10-point summary；
- one-page brief；
- infographic；
- topic map。

---

5. 建議第一版使用五輪分析

不要：

> upload → final report

而是強制 multi-pass。

Round 1 — Corpus Reconstruction

輸出：

```
01_corpus_map.md
02_timeline.md
03_document_genealogy.md
04_initial_claim_index.md
```

只回答：

> 有甚麼？

Round 2 — Research-State Reconstruction

輸出：

```
05_claim_state_map.md
06_branch_map.md
```

```
07_intervention_ledger.md
08_residual_ledger.md
09_nogo_ledger.md
10_model_corrections.md
```

回答：

> 發生了甚麼變化？

Round 3 — Critical Dynamics Analysis

輸出：

```
11_critical_transition_atlas.md
12_failure_and_gap_audit.md
13_hidden_results.md
14_overlooked_branches.md
```

回答：

> 甚麼真正重要？

Round 4 — Discovery / Opportunity Analysis

輸出：

```
15_collision_graph.md
16_latent_opportunities.md
17_specialist_referral_map.md
18_breakthrough_frontiers.md
```

回答：

> 還有甚麼沒有被看見？

Round 5 — Intervention / Replay

輸出：

```
19_replay_candidates.md
20_counterfactual_interventions.md
21_replay_prompts.md
22_next_research_plan.md
23_experiment_candidates.md
```

回答：

> 下一步應怎樣重新介入？

---

6. 用戶不需要每次跑全部功能

這一點最好從一開始設計。

例如：

Quick Audit

只跑：

```
Corpus Map
Residual Ledger
Critical Transitions
Next Actions
```

Residual Audit

只輸出：

```
Residual Ledger
False Closure
Residual Persistence
Suggested Resolution
```

Breakthrough Audit

只輸出：

```
Critical Nodes
Hidden Results
Dormant Branches
Breakthrough Frontiers
```

Executive Audit

只輸出：

```
Major Changes
Major Risks
Unresolved Issues
High-value Opportunities
Recommended Actions
```

Full Research Dynamics Audit

全部跑。

---

7. 建議底層 Research Object Model

下一個 Session 應優先研究這部分。

至少包括：

```
Event
Claim
Beam
Constraint
Residual
```

```
FailedMapping
Intervention
NoGo
Branch
Evidence
Transformation
ResearchState
Document
Actor
Source
```

Actor 至少：

```
Human
Model
Tool
ExternalEvidence
Joint
```

---

8. Human Intervention Taxonomy

可以先採用目前版本：

```
BeamAdd
ResidualFlag
ConstraintAdd
Reframe
Downgrade
Reject
BranchSelect
MethodChange
NoGoCommit
Refactor
Commit
```

但下一個 Session 可以再檢查是否需要：

- QuestionShift
- EvidenceDemand
- SourceInjection
- BlindnessImposition
- CounterexampleDemand
- ScopeChange
- Freeze / Commit

---

9. Model Contribution Taxonomy 也應另外建立

不要只分析 Human。

例如：

```
Expansion
Formalization
Compression
Counterexample
Correction
Downgrade
ConstraintDetection
ResidualDetection
```

```
AlternativeGeneration
Synthesis
Overreach
PrematureClosure
```

尤其要區分：

> **Model-initiated correction**

和：

> model merely complying with a human correction。

---

10. 必須寫入 Kernel 的 epistemic rules

例如：

> Candidate Generation ≠ Claim Validation
>
> Compatibility ≠ Necessity
>
> Formalization ≠ Evidence
>
> Observed Recurrence ≠ Independent Recurrence
>
> Model Self-Correction ≠ Self-Awareness
>
> Rejected ≠ Deleted
>
> Residual ≠ Noise
>
> Later Than ≠ Derived From
>
> Semantic Similarity ≠ Causal Influence
>
> Long Context ≠ Complete Reconstruction

這些不是口號。

是 analysis gate。

---

11. 初步最可能使用者

這應該定位為偏高端工具。

A. Independent Researchers

尤其長期與 ChatGPT / Claude 合作：

- theory building；
- philosophy；
- mathematics；
- AI；
- interdisciplinary work。

B. PhD / Academic Researchers

有：

- 幾十至幾百 AI sessions；
- literature exploration；
- hypothesis evolution；
- drafts；
- rejected ideas。

痛點：

> 已經無法知道自己的 intellectual trajectory。

C. AI-for-Science / AI Research Teams

例如：

- hypothesis agents；
- coding agents；
- research agents；
- multi-agent systems。

用途：

> 分析 research trajectories，而不只 agent logs。

D. AI Safety / Alignment

特別適合：

- long-horizon agent behavior；
- human intervention effect；
- model correction；
- attractor lock-in；
- branch dependence。

E. Frontier Labs / R&D Teams

例如：

```
20 researchers
×
many agents
×
many branches
×
months
```

需要：

> **Research Observability**

不只是：

> agent observability。

F. Senior R&D / Research Management

他們可能不想讀 raw corpus。

但需要知道：

- intellectual position 怎樣改變；
- 最大 unresolved issues；
- 哪個 decision 改變最多；
- 下一步該投入哪裏。

G. Technical Founders

長期使用 AI 設計：

- architecture；
- product；
- protocol；
- strategy。

可以分析：

> design genealogy。

---

12. Skill 解決的現有痛點

最核心不是「聊天太多」。

而是：

Research Context Explosion

AI 能產生的 research material 開始超過人類 sensemaking 能力。

典型症狀：

1. History overload

> 我和 AI 做了半年，已不知道談過甚麼。

---

2. Important node burial

真正重要的一句話埋在幾十萬 tokens 裏。

---

3. Final-output bias

人只看到最後文章。

不知道：

- 哪些 branch 死過；
- 哪些 objection 改變理論；
- 哪些 correction 是關鍵。

---

4. Lost Residuals

AI 很擅長產生 closure。

未解問題容易被漂亮 prose 掩蓋。

---

5. Repeated mistakes

因為 rejected mapping / No-Go 沒保存，後來又重新犯。

---

6. False recurrence

同一 vocabulary 不斷重現，被誤認為 independent support。

---

7. Premature branch closure

當時放棄的 idea 後來可能重新有價值。

---

8. Human intervention invisibility

使用者往往不知道：

> 自己哪一次提問真正改變了後面的研究空間。

---

9. Management opacity

高層知道：

- 做了多少 experiments；
- 用了多少 agents；

但不知道：

> intellectual position 改變了甚麼。

---

10. No replay mechanism

即使知道歷史上有關鍵點，也沒有系統方法問：

如果當時換一個 intervention，會怎樣？

---

13. 它真正要建立的新類別

我現在最傾向：

Research Observability

或：

Epistemic Observability

普通 observability：

看見 machine execution。

Research Observability：

看見 **intellectual evolution**。

核心問題：

```
What was believed?
What changed?
Why did it change?
What failed?
What remains unresolved?
What was overlooked?
Where should we intervene next?
```

---

14. 未來可能開拓的應用

A. Research Governance

不只是找 idea，而是管理：

- candidate；
- residual；
- No-Go；
- evidence；
- branch；
- experiment。

B. Counterfactual Research Replay

真正開始做：

```
G(S + O)
G(S − O)
G(S + Sham(O))
```

研究：

哪類 intervention 真正改變 research trajectory。

C. Human–AI Collaboration Science

把人–AI collaboration 本身變成 experimental object。

D. Cross-Researcher Comparison

例如：

```
Researcher A
Researcher B
Researcher C
```

比較：

- intervention styles；
- residual handling；
- branch discipline；
- model dependence。

E. Independent Recurrence Detection

不同 corpus：

```
Corpus A → I_A
Corpus B → I_B
Corpus C → I_C
```

blind compare。

這可以真正支援 Semantic Collider validation。

F. Lab-level Research Intelligence

例如：

```
Team-level Research State Graph
```

不是普通 project management。

而是：

> intellectual project management。

G. AI Agent Evaluation

不只問 agent：

> task success？

而是：

> 是否產生 productive research evolution？

H. Research Memory Infrastructure

未來 agent 可以讀：

```
Claims
Residuals
No-Go
Branches
Evidence
Interventions
```

而不是重新讀全部 conversation。

I. Specialist Referral

Skill 可以自動指出：

> 這段 research fragment 可能值得哪個領域專家看。

這對你的 SMFT / 成界之學 sample 特別有傳播價值。

---

15. 第一版 **Project Skill** 的定位不要太大

我建議第一版只證明三個 Killer Jobs：

Recover

> 幫我重建真正發生過的研究變化。

Audit

> 告訴我甚麼錯了、漏了、未解、過早關閉。

Redirect

> 告訴我最值得重開、replay 和繼續的地方。

如果這三件已經顯著優於普通 summary，

Skill 就成立。

---

16. 下一個 **Session** 應首先完成的工作順序

不要直接寫最終 SKILL。

先做：

Step 1 — Corpus Packaging Specification

正式定義：

> 用戶要給甚麼。

Step 2 — Research Object Model

正式定義：

> Skill 眼中的世界由甚麼 objects 組成。

Step 3 — Analysis Function Catalogue

正式定義：

> 有哪些可選分析功能。

Step 4 — Intermediate Artifact Catalogue

正式定義：

> 每一步輸出甚麼。

Step 5 — Five-Round Default Protocol

正式定義：

> full analysis 怎樣運行。

Step 6 — Quality / Epistemic Gates

防止：

- overclaim；
- retrospective invention；
- answer contamination；
- attractive but unsupported synthesis。

Step 7 — Sample Regression Test

用：

$\mathbb{O} \to G_2/SO(4) \to \mathbb{H} \to \mathbb{C}^2$ 1–23

測試。

Step 8 — Project System Prompt

只負責 orchestration。

Step 9 — `SKILL.md`

把以上正式編譯成可執行 Skill。

可直接貼到下一個 **Session** 的開場 **Prompt**

> 我想在這個 Session 正式設計一個 **Human–AI Research Dynamics Kernel — Project Edition**。
>
> 它首先假設使用環境是一個類似 GPT-5.6 Sol + Project 的強大 LLM：可以存取 Project Documents、讀取使用者上傳的大型 `.tar` research corpus、進行多輪分析，並以多份中間分析文件逐步建立 Research State；目前先不處理 Codex / Claude Code 的 chunking、checkpoint 和 context engineering，第二階段才把已證明有效的 Project Skill 移植過去。
>
> 請先閱讀 Project 中與下列內容相關的文件：
>
> 1. `The Semantic Collider`
> 2. `Reconstructable Research`
> 3. `From Dialogue to Research Architecture: How Long-Horizon Human–AI Collaboration Revises, Filters, and Distills Theory`
> 4. `From Dialogue to Research Architecture — Short Version`
> 5. Sample raw corpus：`𝕆 → G₂/SO(4) → ℍ → ℂ² 成界過程初探 1–23`
> 6. Bridge corpus：與 `ChatGPT 5.6` 探討「SMFT」、「成界之學」開宗立派還欠缺的準備材料
> 7. 四份 World-Formation 英文綱領：Research Programme、Formal Core、Experimental Programme、E4 Preregistration。
>
> 注意：**Sample raw corpus** 應視為 **Natural-History Dataset**；**Bridge document** 是 **meta-analysis / annotation layer**；四份英文綱領是 **Gold Distillation Outputs**。**Blind analysis mode** 不應先讀 **Gold outputs**。
>
> 這個 Skill 的核心不是 conversation summarization，而是：
>
> **Reconstruct → Diagnose → Discover → Intervene**
>
> 它應回答：
>
> 1. What changed?
> 2. What caused the change?
> 3. What failed or remains unresolved?
> 4. What was overlooked?
> 5. Which nodes are worth replaying?
> 6. What should happen next?
>
> 預期主要研究物件包括：
>
> `Event, Claim, Beam, Constraint, Residual, FailedMapping, Intervention, NoGo, Branch, Evidence, Transformation, ResearchState, Actor, Source.`
>
> 初步 Human Intervention Taxonomy 包括：

`BeamAdd, ResidualFlag, ConstraintAdd, Reframe, Downgrade, Reject, BranchSelect, MethodChange, NoGoCommit, Refactor, Commit.`

需要研究並設計的主要分析功能包括但不限於：

- Corpus Map
- Research Timeline
- Claim State Map
- Branch Map
- Lineage / Genealogy
- Residual Ledger
- Failed Mapping Register
- Error / Gap Audit
- No-Go Ledger
- Attractor / Lock-in Audit
- Critical Transition Atlas
- Hidden Results
- Overlooked Branches
- Breakthrough Frontiers
- Latent Collision Opportunities
- Human Intervention Analysis
- Model-Initiated Correction Analysis
- Specialist Referral Map
- Replay Candidate Detection
- Counterfactual Intervention
- Suggested Re-discussion Prompts
- Next Research Plan
- Experiment Candidates
- Executive / Reviewer / Scientist / Public projections

其中很多功能應可以單獨執行；例如只輸出一份高質量 **Residual Ledger** 本身已可能非常有價值。

Full mode 初步考慮五輪：

Round 1 — Corpus Reconstruction
Round 2 — Research-State Reconstruction
Round 3 — Critical Dynamics Analysis
Round 4 — Discovery / Opportunity Analysis
Round 5 — Intervention / Replay

第一版最重要的三個 Killer Jobs 是：

Recover — 重建真正發生過的研究演化。
Audit — 找出錯誤、漏洞、Residual、No-Go、過早關閉。
Redirect — 找出最值得重開、replay 和繼續的節點。

長遠定位可能是：

Research Observability / Epistemic Observability for long-horizon Human–AI research.

它試圖解決的核心痛點是 **Research Context Explosion**：AI 已可以產生大量長期研究材料，但人類越來越難知道自己的研究到底怎樣演化、哪個 intervention 真正重要、哪些問題仍未解、哪些 branch 被錯誤關閉，以及下一步最值得在哪裏重新介入。

這個 **Session** 先不要直接寫 **SKILL.md**。

第一階段請依次完成：

1. Corpus Packaging Specification
2. Research Object Model
3. Analysis Function Catalogue
4. Intermediate Artifact Catalogue
5. Five-Round Analysis Protocol
6. Epistemic / Quality Gates
7. Sample Regression Strategy

然後才研究如何把它編譯成 Project System Prompt + Kernel files + `SKILL.md`。

請先閱讀相關 Project Documents，重新檢查以上架構是否有遺漏、重複或分類錯誤，然後提出 **v0.1 architecture plan**；這一輪先不要生成最終 Skill。

這樣開新 Session，基本上就能從我們現在的位置直接接下去，而不是重新花幾輪才重新發現「這個 Skill 到底是做甚麼的」。

是否還可以有一個功能，就是分析很牛的一些高手對話，指出這批對話裏，最精彩的部分？而且輸出的表達方式/格式，也會很有價值和特色！我覺得單單是這個功能，就很有教育意義，也因此會有很多人學習這個 **Skill** 的用法！
若你也同意，請補充一個關於這個功能的指引，給下一個 **Session** 參考。

我同意，而且我覺得這個功能很可能比「完整 **Research Dynamics Audit**」更容易被普通人理解和傳播。

因為用戶不必先明白 Residual、No-Go、Lineage、Counterfactual Replay 是甚麼；他只需要問：

「這幾百頁高手對話裏，到底哪幾段最精彩？為甚麼精彩？我可以學甚麼？」

這已經是一個非常直觀的需求。

更重要的是，它不是普通的「精華摘要」。如果做好，應該分析：

高手在甚麼節點做了甚麼高價值 **intellectual move**，為甚麼這個 **move** 改變了後面的研究，以及別人怎樣學會這種 **move**。

我會把這個功能暫名為：

Intellectual Highlight Mining

或更有特色一點：

Master Dialogue Atlas

中文可以叫：

高手對話精華圖譜

它可以成為整套 Skill 很好的教育入口。

它不是找「最好看的句子」

這點下一個 Session 一定要寫清楚。

普通 highlight extractor 很容易挑：

- 文筆漂亮；
- 聽起來深刻；
- 金句；
- 戲劇性回答。

但這個 Skill 應該找的是：

> 高信息密度、高研究價值、高可遷移性的 **intellectual moves**。

例如：

- 一句話推翻了錯誤前提；
- 一個問題令整個研究轉向；
- 一個 residual 被準確指出；
- 一個漂亮理論被主動 downgrade；
- 一個 AI correction 防止了大規模 overclaim；
- 一個陌生 beam 恰好解開長期問題；
- 一個方法論改變令後面可以 blind derive；
- 一個 apparently minor objection 後來成為整條研究線的起點；
- 一個很好的 compression，把數十頁討論變成一個可操作 structure。

這才是「精彩」。

可以增加一套 **Highlight Taxonomy**

下一個 Session 可以先研究這些類別是否足夠。

1. Breakthrough Move

真正打開新方向。

2. Reframing Move

不是回答舊問題，而是改變問題本身。

3. Residual Detection

指出大家一直忽略的未解問題。

4. Error Kill

一句或一段把漂亮但錯誤的方向終止。

5. Constraint Injection

加入一條限制，令後面的理論突然收斂。

6. Beam Injection

引入一個遠方概念，改變整個 search space。

7. Theory Downgrade

主動把「宇宙真理」降成「工具／probe／interpretation」。

8. Compression

把龐大複雜結構壓縮成簡潔而不失真的 representation。

9. Synthesis

把原本分散的 threads 組成新的 coherent architecture。

10. Methodological Upgrade

例如由自由探索改成 blind derivation、ablation、preregistration。

11. Model-Initiated Correction

AI 主動指出 earlier model / human assumption 有問題。

12. Human Search-Space Governance

人類不是提供答案，而是改變後面可以搜尋甚麼。

13. Productive Failure

失敗本身直接產生下一個理論。

14. Dormant Gem

當時沒有發展，後來看才發現很重要。

15. Teaching-Quality Exchange

一段對話非常適合讓別人學會某種 reasoning move。

「精彩」最好不要只給一個總分

我反而建議每個 Highlight 有多個維度。

例如：

```
Intellectual Value
Research Impact
Surprise
Compression
Transferability
Pedagogical Value
Downstream Influence
Epistemic Discipline
```

不一定要數值化。

可以用：

```
High / Medium / Low
```

或者：

```
Primary strength:
Reframing + Residual Detection + Downstream Impact
```

避免製造假精確度。

每一個精彩節點，輸出格式可以非常有特色

我很推薦做成固定的 **Highlight Card**。

例如：

Highlight 07 — The Four-Phase Downgrade

Original situation
Four phases were beginning to look like a candidate fundamental ontology.

Key move
The human explicitly reframed them as a diagnostic probe for persistent and self-renewing worlds rather than a universal structure.

Why this was exceptional
The move removed an attractive explanatory overreach instead of expanding it.

What changed afterwards
The theory could continue using the four-phase structure without requiring every admissible world to instantiate it.

Move type
`Reframe + Downgrade + ConstraintAdd`

Transferable lesson
When a pattern fits too many things, ask whether it should be a probe rather than an ontology.

Replay exercise
Take one framework you currently treat as fundamental and ask:
“What if this is only an observational or diagnostic coordinate?”

這就不是摘要。

是：

> 對 **intellectual craftsmanship** 的標註。

更好的地方是：可以做「前後對照」

例如：

```
BEFORE
“Maybe this structure is fundamental.”
```

```
CRITICAL MOVE
“What if it is only a diagnostic structure?”

AFTER
Fundamental ontology
→ comparative probe
```

視覺上會很好看。

而且教育價值很高。

還可以有「一句話為甚麼牛」

這會很適合 social sharing。

例如：

> **Why this move matters:**
> It improved the theory by reducing what it claimed.

或者：

> **The clever move was not finding a new answer, but changing what counted as a valid question.**

或者：

> **This intervention removed an entire family of future overclaims.**

這些比長文章容易傳播很多。

可以再做一個「高手做了甚麼」層

例如分析整批對話後，輸出：

Recurring Expert Moves

這位研究者／對話系統反覆使用：

1. 不滿足於 coherent answer，追問 hidden assumption。
2. 把失敗保存成 residual，而不是忘掉。
3. 故意加入異質 conceptual beam。
4. 當理論太漂亮時主動要求反證。
5. 把 ontology 降為 operational probe。
6. 分離 discovery 與 validation。
7. 在關鍵時刻重新定義問題。
8. 把失敗轉成 No-Go。
9. 要求 AI 重新 blind derive。
10. 把 insight 編譯成 experiment。

這會非常有教育價值。

因為讀者開始學的不是：

> 「他得出了甚麼答案？」

而是：

> 「他怎樣思考？」

甚至可以建立「**Dialogue Move Library**」

這可能成為非常有吸引力的長期產品。

例如從不同高手 corpus 抽出：

```
Move #014 — Ontology → Probe
Move #021 — Residual → New Beam
Move #034 — Attractive Theory → Ablation Test
Move #041 — Local Objection → Global Reframe
Move #057 — Model Correction → No-Go
```

每個 Move 附：

- 原始案例；
- 抽象形式；
- 何時使用；
- 不適用條件；
- 一個 replay prompt；
- 一個練習。

最後可以形成：

Human–AI Research Move Library

這本身可能比一個長理論文章更容易被人學習和分享。

還可以設一個「**Top 10 Intellectual Moments**」輸出

這很適合 public-facing。

但不是：

> Top 10 most dramatic quotes

而是：

> Top 10 moments that most changed the research trajectory.

每個只佔半頁甚至一張卡：

```
Moment
Context
Move
Why it mattered
Downstream effect
What to learn
```

這可能就是你說的：

> 「單單這個功能就有教育意義。」

我同意。

可以再做三種不同的 **Highlight Mode**

1. Research Highlight Mode

找對研究最重要的節點。

2. Learning Highlight Mode

找最值得學生模仿的 reasoning moves。

3. Public Highlight Mode

找最容易讓外部人看懂「這批對話為甚麼不普通」的片段。

同一 corpus，三者不一定挑同樣的地方。

這個區分很重要。

Public Highlight Mode 特別值得做

因為它可以解決：

> 「950 頁 Sample 誰會看？」

Skill 可以生成：

20 Moments Worth Reading in a 950-Page Human–AI Research Dialogue

每個給：

- 位置；
- 100–300 字 context；
- 關鍵對話；
- why it matters；
- later consequence。

這就把 950 頁變成一條 curated path。

讀者不需要 linear read。

更進一步：**Skill** 可以生成「學習路線」

例如：

If you want to learn research reframing

看：

```
Episode 4
→ Episode 9
→ Episode 16
```

If you want to learn how to challenge AI

看：

```
Episode 8
→ Episode 12
→ Episode 17
```

If you want to learn theory distillation

看：

```
Episode 14
→ Episode 15
→ later programme documents
```

所以 sample corpus 就變成：

> 可導航的研究教科書。

這可能是非常好的副產品。

這個功能對傳播也有特殊作用

原本：

```
950-page corpus
→ nobody reads
```

加入 Highlight Mining：

```
950-page corpus
→ Top 20 moments
→ one moment catches reader
→ reader opens source section
→ discovers related research
→ perhaps forwards to specialist
```

也就是我們之前討論的：

> **Tool-Mediated Incidental Exposure**

會變得更加強。

因為 Skill 主動建立「入口」。

我建議把它正式加入架構，作為第五個功能族

之前是：

```
RECONSTRUCT
DIAGNOSE
DISCOVER
INTERVENE
```

現在可以增加：

ILLUMINATE

回答：

> **What is most worth seeing, learning, and teaching in this corpus?**

這個詞很好，因為它不是單純 summary。

ILLUMINATE 可以包含

E1. Intellectual Highlight Mining

找最精彩節點。

E2. Top Intellectual Moments

排行／精選。

E3. Expert Move Extraction

抽取可泛化的高手思考動作。

E4. Learning Cards

轉成教學卡片。

E5. Curated Reading Path

建立閱讀路線。

E6. Quote + Context

保留重要原文和最小必要上下文。

E7. Before / Move / After

顯示研究狀態變化。

E8. Transferable Lesson

說明可遷移技巧。

E9. Replay Exercise

把原案例變成學習練習。

E10. Public Showcase

把整批 corpus 濃縮成可分享展示。

給下一個 **Session** 的補充指引

你可以直接把以下一段加入我上一個 response 的開場 Prompt：

Additional Function Family — ILLUMINATE / Intellectual Highlight Mining

除了 `Reconstruct → Diagnose → Discover → Intervene`，請研究加入第五個功能族：

ILLUMINATE — identify what is most worth seeing, learning, and teaching in a long Human–AI research corpus.

這個功能不是普通摘要、quote extraction 或「找金句」，而是識別 **high-value intellectual moves**：某些人類或 AI intervention 對研究方向、問題定義、constraint、residual、branch、claim state 或後續發展造成重大影響的節點。

初步 Highlight Types 包括：

- Breakthrough Move
- Reframing Move
- Residual Detection
- Error Kill
- Constraint Injection
- Beam Injection
- Theory Downgrade
- Compression
- Synthesis
- Methodological Upgrade
- Model-Initiated Correction
- Human Search-Space Governance
- Productive Failure
- Dormant Gem
- Teaching-Quality Exchange

每個 Highlight 應盡量重建：

```
Before State → Critical Move → After State → Downstream Consequence
```

並至少輸出：

- Source location
- Minimal context
- Critical move
- Why it matters
- What changed afterwards
- Move type

- Transferable lesson
- Confidence / uncertainty

可選增加：

- Replay exercise
- Suggested prompt for practising the same move
- Related later nodes
- Similar moves elsewhere in the corpus

不應以文筆漂亮、戲劇性或聽起來深刻作為主要選擇標準；主要標準應是：

- Research impact
- Epistemic value
- Downstream influence
- Reframing power
- Residual resolution/exposure
- Compression quality
- Transferability
- Pedagogical value

請研究至少三種 Highlight Mode：

Research Highlight Mode — 哪些節點最改變研究軌跡。
Learning Highlight Mode — 哪些節點最值得其他研究者模仿。
Public Highlight Mode — 哪些節點最適合向外部讀者展示這批 corpus 的特殊性。

預期輸出包括：

- `intellectual_highlight_atlas.md`
- `top_intellectual_moments.md`
- `expert_move_library.md`
- `learning_cards.md`
- `curated_reading_path.md`
- `public_showcase.md`

特別研究一種標準 **Highlight Card** 格式：

```
Context → Key Move → Why Exceptional → Downstream Effect → Transferable Lesson → Replay Exercise
```

並考慮從多個高質量 corpus 長期建立 **Human–AI Research Move Library**，把具體高手對話抽象成可教、可模仿、可 replay 的研究技巧。

此功能可能同時承擔三個角色：

1. **Education** — 教人如何進行高質量 Human–AI research interaction；
2. **Navigation** — 讓人不用線性閱讀巨大 corpus，也能找到最值得看的地方；
3. **Dissemination** — 透過精彩節點把讀者引回 Sample raw corpus，形成 specialist referral 和 incidental exposure。

請把 ILLUMINATE 視為與完整 Research Dynamics Audit 不同但共享底層 Research State 的 projection / analysis family，而不是單純在 final report 上增加一個「Highlights」章節。

我甚至覺得這個功能可能會成為整套工具的最佳入口功能：

> **Full Audit** 是專業功能；**Highlight Atlas** 是讓人第一次體會這套方法有甚麼不同的功能。

很多人未必一開始想做 Research Observability，但他很容易想：

> 「幫我看看這 **500** 頁高手對話到底哪裏最厲害，以及我可以偷學甚麼。」

而一旦他看見 Skill 能抓到那些普通 summary 完全看不到的 intellectual moves，他自然就會開始理解為甚麼還需要 Residual Ledger、Critical Transition Atlas、Replay Analysis 等更深層功能。

但我聯想到的，就是在 **YouTube** 看別人打機，我們配個精彩旁述！寫得好，肯定有需求。

你不用長回應。但幫我找找目前是否有已經有這種 **Skill**，我將會寫的，會比目前有的更好嗎？

有接近的，但我剛查了一輪，暫時沒找到和你想做的完全同類產品。

目前公開工具主要落在三類：一類是 ChatGPT/Claude 歷史搜尋、topic clustering、timeline，例如 AI Chat Explorer、ChatLens、ChatInsights；一類是 conversation analysis，例如 `gpt-chat-analysis`，會分析 decision loop、協作模式、成功率等；另一類是 meeting/highlight 工具，例如 Fireflies、Fathom、Wayin 等，會抽取 key moments、decisions、action items、shareable clips。GitHub

但你構想的功能更像：

> 「把高手長對話當成一場精彩比賽，**AI** 不只剪精華，還做高水平戰術旁述。」

它不是單純說「這裏很重要」，而是解釋：

> 這一問為什麼厲害 → 它改變了什麼 search space → AI 怎樣接招 → 哪個 residual 被逼出來 → 後面幾十輪因此怎樣改變 → 其他人可以學哪一招。

我目前找到的工具，多數停在 **highlight / summary / decision / topic**；沒有明顯做到你前面設計的 **Before State → Critical Move → After State → Downstream Consequence → Transferable Lesson → Replay Exercise** 這一層。即使最接近的 conversation analyzer，也主要按既定 decision-loop 框架評估互動，而不是抽取「intellectual craftsmanship」。GitHub

所以，若你的 **Skill** 真能穩定做到這件事，我認為會明顯比目前公開可見的同類工具高一層。尤其真正差異不是模型更會摘要，而是你有 Semantic Collider + Reconstructable Research + Research Dynamics 那套底層 object model，能判斷一段對話精彩在「哪種研究動作」。

你那個 YouTube 打機比喻其實很準：

> 普通工具 = 自動剪出擊殺精華。
> 你的 Skill = 剪出關鍵局，然後由懂戰術的人告訴你：這裏為甚麼是高手操作，以及如果換一手會發生甚麼。

這個功能我會很認真地保留，甚至可能成為第一個最容易讓外界理解和試用的 **showcase feature**。

© 2026 Danny Yeung. All rights reserved. 版权所有 不得转载

Disclaimer

This book is the product of a collaboration between the author and OpenAI's GPT 5.6, Google AI, Gemini 3.X, NoteBookLM, X's Grok, Claude' Sonnet 5 language model. While every effort has

been made to ensure accuracy, clarity, and insight, the content is generated with the assistance of artificial intelligence and may contain factual, interpretive, or mathematical errors. Readers are encouraged to approach the ideas with critical thinking and to consult primary scientific literature where appropriate.

This work is speculative, interdisciplinary, and exploratory in nature. It bridges metaphysics, physics, and organizational theory to propose a novel conceptual framework—not a definitive scientific theory. As such, it invites dialogue, challenge, and refinement.

I am merely a midwife of knowledge.